Maximum flow

Outline

The maximum flow problem

Cuts

The residual network

The max-flow min-cut theorem

The Ford–Fulkerson algorithm

The Edmonds–Karp algorithm

Applications

Typical mistakes

In summary

Learning objectives

At the end of this chapter you will be able to:

- state the maximum flow problem and write its LP model;
- compute the capacity of an s–t cut, and use cuts as upper bounds on the value of every flow;
- build the residual network of a flow, find an augmenting path and its bottleneck, and augment the flow;
- prove the max-flow min-cut theorem and the integrality theorem;
- run the Ford–Fulkerson and the Edmonds–Karp algorithms, read a minimum cut at the stop, and give their running times;
- compute a maximum matching in a bipartite graph and a maximum set of arc-disjoint paths as maximum flows.

Prerequisites

- From this module, Chapter 1 «Graphs»: directed paths, the breadth-first search and the depth-first search.
- From this module, Chapter 4 «Network flow models»: flow conservation, the flow across a cut, the minimum cost flow model and its integer vertices.
- From the module Linear Programming, Chapter 15 «The geometry of Linear Programming»: a linear function attains its maximum over a nonempty bounded polyhedron.

The maximum flow problem

Introduction: The maximum flow problem

- In this section we define the maximum flow problem: send as much flow as possible from a source s to a sink t, respecting the capacities of the arcs
- It is one of the basic problems of network optimization: it models pipelines, traffic, communication networks, and many assignment problems
- We introduce the reference network of the chapter and write the problem as a Linear Programming model, the MF model

Definition 1: maximum flow problem (1/2)

*Let $G = (N, A)$ be a directed graph with $n = |N|$ nodes and $m = |A|$ arcs, two distinct nodes s, the **source**, and t, the **sink**, and a **capacity** $u_{ij} \geq 0$ for every arc $(i, j) \in A$.*

- *A **feasible flow** is a vector $\boldsymbol{x} \in \mathbb{R}^m$ with $0 \leq x_{ij} \leq u_{ij}$ for every arc $(i, j) \in A$ and flow conservation at every node other than s and t:*

$$\sum_{(i,j)\in\delta^+(i)} x_{ij} - \sum_{(j,i)\in\delta^-(i)} x_{ji} = 0, \quad \forall i \in N \setminus \{s, t\}.$$

Definition: maximum flow problem (2/2)

- *The **value** of the flow is the net outflow of the source:*

$$v = \sum_{(s,j)\in\delta^+(s)} x_{sj} - \sum_{(j,s)\in\delta^-(s)} x_{js}.$$

- *The **maximum flow problem** asks for a feasible flow of maximum value, a **maximum flow**. Its value is denoted by $\tilde{v}$.*

Remarks on the definition

- The capacities are finite, and the flow $\boldsymbol{x} = 0$, of value 0, is always feasible.
- The nodes other than s and t pass on everything they receive: the flow leaves s and travels through the network towards t.
- As in Chapter 4 «Network flow models», $\delta^+(i)$ and $\delta^-(i)$ are the arcs leaving and entering node i, and the arcs are numbered $a_1, a_2, \ldots, a_m$ in lexicographic order.

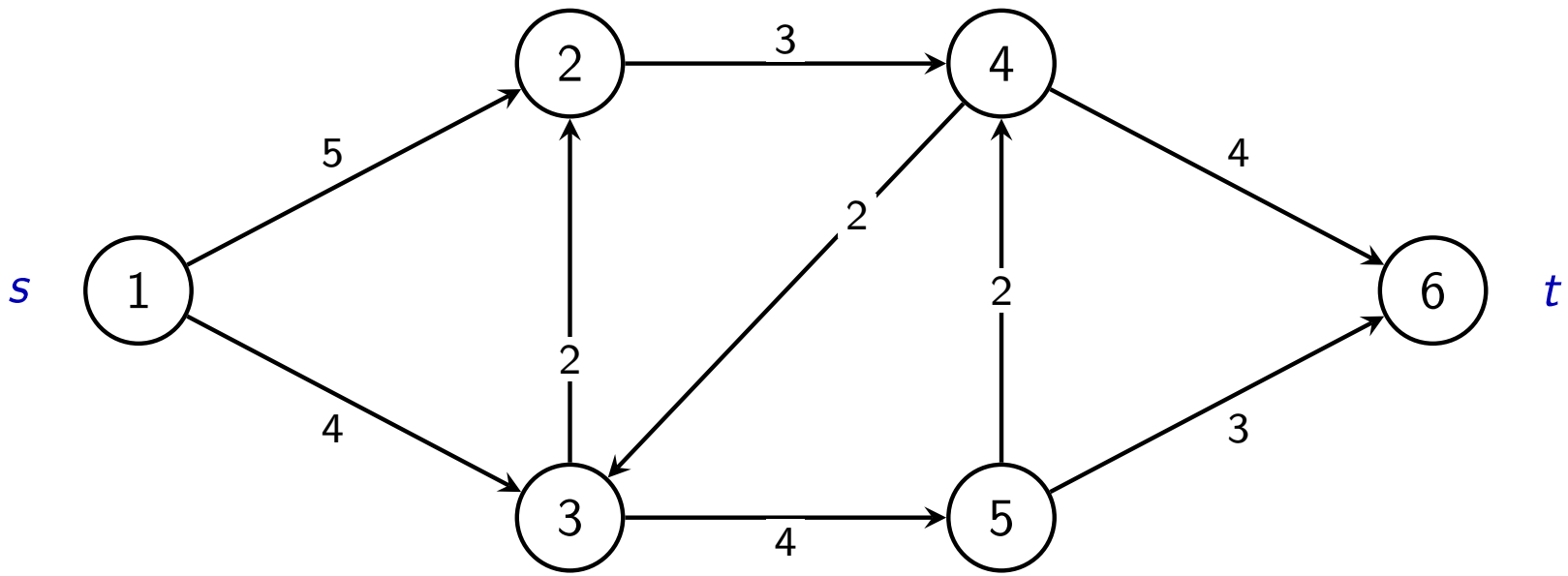
Example: the reference network (1/4)

- We give capacities to the reference graph of Chapter 1 «Graphs», with source $s = 1$ and sink $t = 6$
- The table also gives a flow $\bar{\boldsymbol{x}}$, which we use in the whole chapter

arc	$(1,2)$	$(1,3)$	$(2,4)$	$(3,2)$	$(3,5)$	$(4,3)$	$(4,6)$	$(5,4)$	$(5,6)$
u_{ij}	5	4	3	2	4	2	4	2	3
$\bar{x}_{ij}$	3	0	3	0	2	2	1	0	2

Example: the reference network (2/4)

In the figure each arc carries its capacity u_{ij}.



Example: the reference network (3/4)

- The flow $\bar{\boldsymbol{x}}$ of the table is feasible: it respects the capacities, and it is conserved at the nodes $2, 3, 4, 5$
- For example, at node 4:

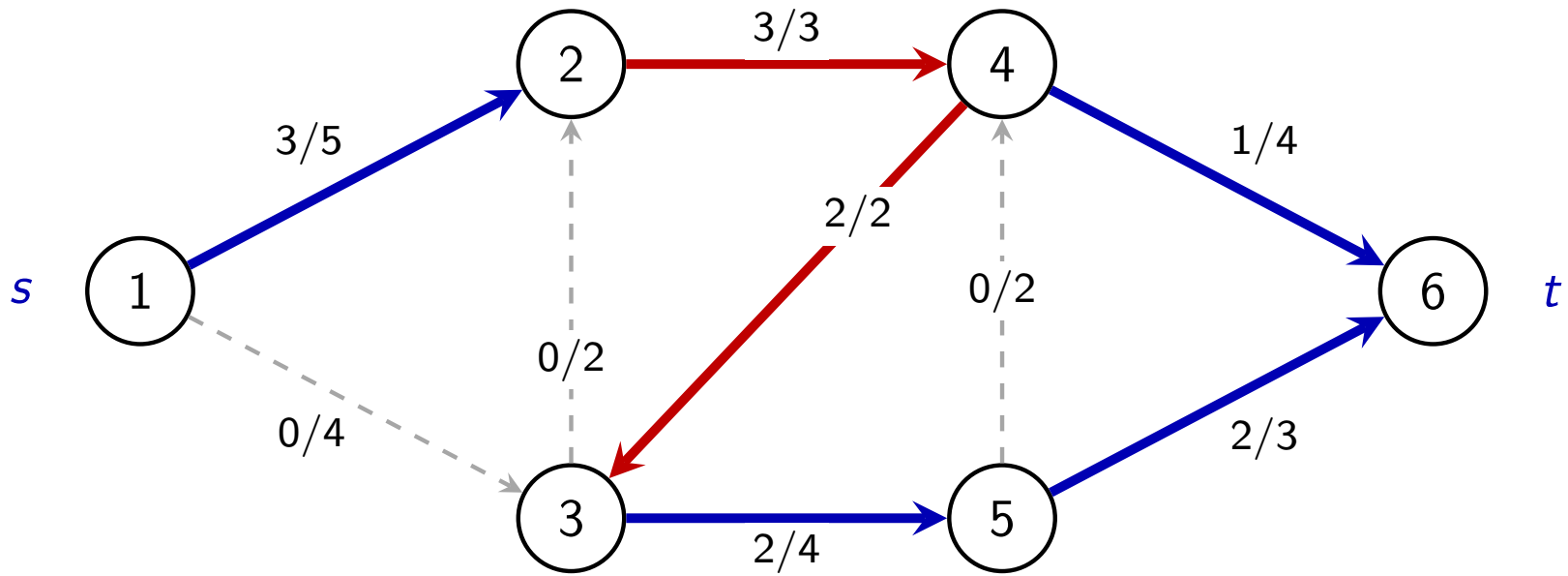
  inflow $\bar{x}_{24} + \bar{x}_{54} = 3 + 0 = 3$, outflow $\bar{x}_{43} + \bar{x}_{46} = 2 + 1 = 3$
- Its value is

$$v = \bar{x}_{12} + \bar{x}_{13} = 3 + 0 = 3,$$

  and node 6 receives $\bar{x}_{46} + \bar{x}_{56} = 1 + 2 = 3$ units

Example: the reference network (4/4)

In the figure each arc carries $\bar{x}_{ij}/u_{ij}$: the arcs with $0 < \bar{x}_{ij} < u_{ij}$ are blue, the saturated arcs ($\bar{x}_{ij} = u_{ij}$) red, and the empty arcs grey and dashed.



The variables of the LP model

- We now write the maximum flow problem as a Linear Programming model
- We use one continuous variable x_{ij} for every arc $(i, j) \in A$, the flow on the arc
- and one variable v, the value of the flow

The MF model

Using these variables, an LP model for the maximum flow problem reads as follows:

$$\max \quad v$$

$$\text{subject to} \quad \sum_{(s,j)\in\delta^+(s)} x_{sj} - \sum_{(j,s)\in\delta^-(s)} x_{js} = v,$$

$$\sum_{(i,j)\in\delta^+(i)} x_{ij} - \sum_{(j,i)\in\delta^-(i)} x_{ji} = 0, \qquad \forall i \in N \setminus \{s,t\},$$

$$\sum_{(t,j)\in\delta^+(t)} x_{tj} - \sum_{(j,t)\in\delta^-(t)} x_{jt} = -v,$$

$$x_{ij} \le u_{ij}, \qquad \forall (i,j) \in A,$$

$$x_{ij} \ge 0, \qquad \forall (i,j) \in A,$$

$$v \in \mathbb{R}.$$

This LP model is referred to as the MF model.

The objective function and the constraints

Description of the objective function and constraints of the MF model:

- the objective function maximizes the value of the flow;
- the constraint of s states that the net outflow of s is v, and the constraint of t that the net inflow of t is v;
- the constraints of the nodes $i \in N \setminus \{s, t\}$ are the flow conservation constraints of the other nodes;
- the constraints $x_{ij} \leq u_{ij}$ impose the capacities, and the constraints $x_{ij} \geq 0$ and $v \in \mathbb{R}$ define the variables of the LP model.

Two remarks on the MF model

- Every arc contributes $+x_{ij}$ to the row of its tail and $-x_{ij}$ to the row of its head, so the left-hand sides of the conservation constraints (those of s, of the other nodes and of t) add up to zero: the constraint of t follows from the others.

- In Chapter 4, Section «Special cases» the maximum flow problem is a minimum cost flow model: zero balances and zero costs, plus the arc (t, s) with capacity $+\infty$ and cost -1.
- Calling v the flow x_{ts} on this arc, the constraints of s, of the other nodes and of t are its conservation constraints, and $\min(-x_{ts})$ is $\max v$.

Feasible flows and the MF model

- A feasible flow respects the capacities and is conserved at every node other than s and t
- Its value v is the net outflow of s
- The maximum flow problem is the LP MF: maximize v under conservation and capacities

Summary

The problem:

- A feasible flow satisfies $0 \leq x_{ij} \leq u_{ij}$ and flow conservation at the nodes other than s and t
- Its value v is the net outflow of the source; a maximum flow has the largest value $\tilde{v}$

The model:

- The MF model maximizes v; the constraint of t follows from the others
- It is a minimum cost flow model with the extra arc (t, s) of cost -1

The reference network: $s = 1$, $t = 6$, and the flow $\bar{\boldsymbol{x}}$ of value 3

Cuts

Introduction: Cuts

- In this section we split the nodes into two sides, one with s and one with t: an s–t cut
- Every unit of flow from s to t must cross the cut, so the capacity of a cut bounds the value of every flow
- These bounds are the key to recognize a maximum flow: they are the other half of the max-flow min-cut theorem

Definition 2: s–t cut

*Let $S \subseteq N$ with $s \in S$ and $t \notin S$. The s–t **cut** $(S, N \setminus S)$ is the partition of the nodes into S and $N \setminus S$.*

- *Its **forward arcs** are the arcs of $\delta^+(S)$, from S to $N \setminus S$, and its **backward arcs** the arcs of $\delta^-(S)$, from $N \setminus S$ to S.*
- *Its **capacity** is the total capacity of the forward arcs:*

$$u(S, N \setminus S) = \sum_{(i,j) \in \delta^+(S)} u_{ij}.$$

- *A **minimum cut** is an s–t cut of minimum capacity.*

Remarks on cuts

- The capacities of the backward arcs do not enter $u(S, N \setminus S)$.
- Each node other than s and t can be on either side, so a network has 2^{n-2} s–t cuts.
- As in Chapter 4, for a set of arcs F we write

$$x(F) = \sum_{(i,j) \in F} x_{ij}.$$

The next lemma shows that the net flow across every s–t cut is the same: the value v of the flow.

Lemma 1: flow across an s–t cut

Let $\boldsymbol{x}$ *be a feasible flow of value* v*. Then, for every* s–t *cut* $(S, N \setminus S)$*,*

$$v = x(\delta^+(S)) - x(\delta^-(S)).$$

In particular, with $S = N \setminus \{t\}$*, the net inflow of* t *is* v*.*

Proof

- Let b_i be the net outflow of node i under $\boldsymbol{x}$: $b_s = v$ and $b_i = 0$ for $i \notin \{s, t\}$
- Then $\boldsymbol{x}$ satisfies flow conservation with these balances, and Chapter 4, Lemma «flow across a cut» gives

$$x(\delta^+(S)) - x(\delta^-(S)) = \sum_{i \in S} b_i, \quad \forall S \subseteq N$$

1. With $S = N$ no arc leaves or enters S, so $\sum_{i \in N} b_i = 0$ and $b_t = -v$.
2. For an s–t cut, $s \in S$ and $t \notin S$, so $\sum_{i \in S} b_i = b_s = v$.

□

Corollary 1: weak duality

For every feasible flow $\boldsymbol{x}$ *of value* v *and every* s–t *cut* $(S, N \setminus S)$,

$$v \leq u(S, N \setminus S),$$

with equality if and only if $x_{ij} = u_{ij}$ *for every* $(i,j) \in \delta^+(S)$ *and* $x_{ij} = 0$ *for every* $(i,j) \in \delta^-(S)$. *In case of equality,* $\boldsymbol{x}$ *is a maximum flow and* $(S, N \setminus S)$ *is a minimum cut.*

Proof

By the lemma on the flow across an s–t cut, and the bounds $x_{ij} \leq u_{ij}$ on $\delta^+(S)$ and $x_{ij} \geq 0$ on $\delta^-(S)$,

$$v = x(\delta^+(S)) - x(\delta^-(S)) \leq u(S, N \setminus S) - 0.$$

- Equality holds if and only if every bound used is tight, arc by arc.
- In case of equality, every feasible flow $\boldsymbol{x}'$ has value $v' \leq u(S, N \setminus S) = v$, and every s–t cut $(S', N \setminus S')$ has $u(S', N \setminus S') \geq v = u(S, N \setminus S)$.

□

Example: the cuts of the reference network (1/3)

The reference network has $2^4 = 16$ s–t cuts. Next to each set S, the capacity $u(S, N \setminus S)$:

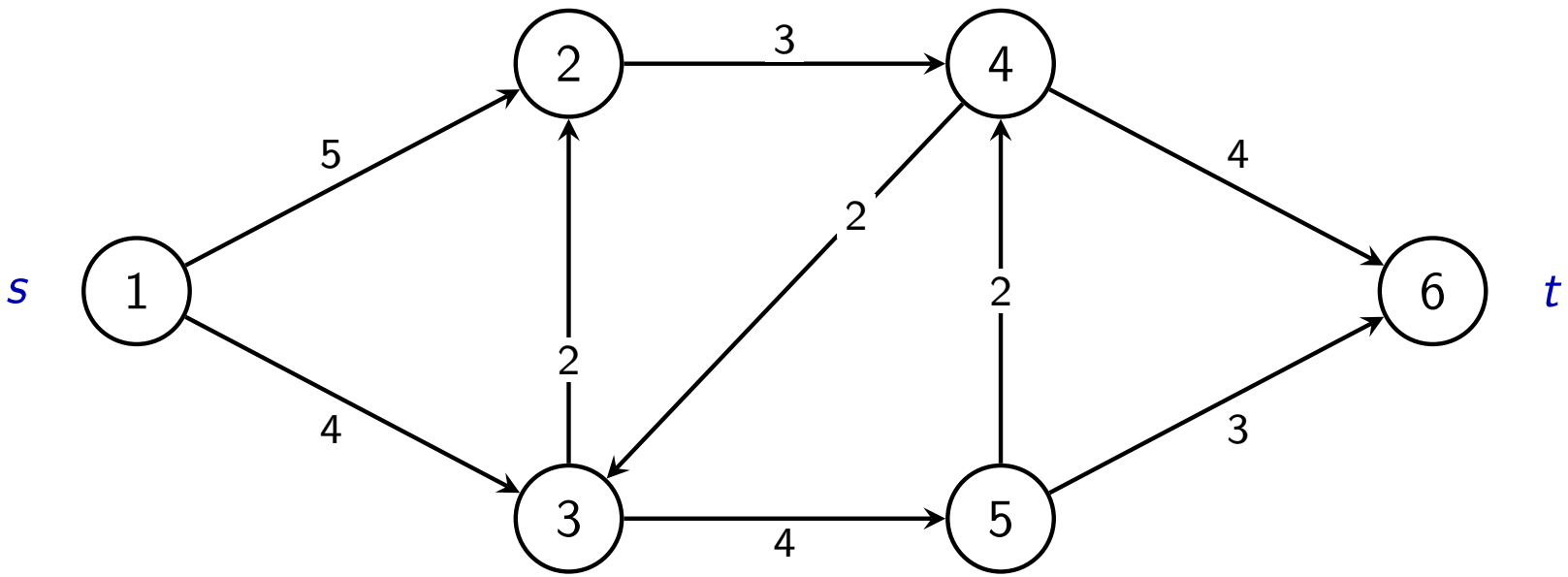
S	u	S	u	S	u	S	u
$\{1\}$	9	$\{1,2\}$	7	$\{1,3\}$	11	$\{1,4\}$	15
$\{1,5\}$	14	$\{1,2,3\}$	7	$\{1,2,4\}$	10	$\{1,2,5\}$	12
$\{1,3,4\}$	15	$\{1,3,5\}$	12	$\{1,4,5\}$	18	$\{1,2,3,4\}$	8
$\{1,2,3,5\}$	8	$\{1,2,4,5\}$	13	$\{1,3,4,5\}$	14	$\{1,2,3,4,5\}$	7

Example: the cuts of the reference network (2/3)

- The minimum capacity is 7, attained by three cuts:

$$S = \{1, 2\}, \qquad S = \{1, 2, 3\}, \qquad S = \{1, 2, 3, 4, 5\}$$

- By the weak duality corollary, no feasible flow has value larger than 7



Example: the cuts of the reference network (3/3)

- For $S = \{1, 2, 3\}$ the forward arcs are $(2, 4)$, $(3, 5)$, and the backward arc is $(4, 3)$
- With the flow $\bar{\boldsymbol{x}}$ of the reference network, the net flow across the cut is the value:

$$\bar{x}_{24} + \bar{x}_{35} - \bar{x}_{43} = 3 + 2 - 2 = 3 = v$$

- and the capacity of the cut is

$$u(S, N \setminus S) = u_{24} + u_{35} = 3 + 4 = 7$$

Observation 1: the capacity of a cut

The capacity $u(S, N \setminus S)$ adds up the capacities of the forward arcs only.

- *For $S = \{1, 2, 3\}$ it is $u_{24} + u_{35} = 7$.*
- *Adding the capacity of the backward arc $(4, 3)$ would give 9, which bounds nothing.*
- *A flow on a backward arc brings units back into S and lowers the net flow across the cut.*

Flows and cuts

- The net flow across every s–t cut is the value v
- Hence $v \leq u(S, N \setminus S)$ for every flow and every cut, with equality when the forward arcs are saturated and the backward arcs empty
- On the reference network no flow has value larger than 7

Summary

Cuts:

- An s–t cut $(S, N \setminus S)$ has $s \in S$ and $t \notin S$
- Its capacity adds up the capacities of the forward arcs only

Flows across cuts:

- The net flow across every cut is $v = x(\delta^+(S)) - x(\delta^-(S))$
- Weak duality: $v \leq u(S, N \setminus S)$; equality certifies a maximum flow and a minimum cut

Reference network: three minimum cuts of capacity 7

The residual network

Introduction: The residual network

- In this section we ask how a feasible flow can be improved
- The residual network collects, arc by arc, how much flow can still be sent in each direction: by raising a flow or by lowering one
- A path from s to t in the residual network, an augmenting path, raises the value of the flow: it is the basic step of all the algorithms of this chapter

An assumption: no opposite arcs

- In the rest of the chapter G has no pair of opposite arcs (i, j) and (j, i), as the reference graph
- An arc (j, i) opposite to (i, j) can be replaced by the arcs (j, k) and (k, i) through a new node k, both with capacity u_{ji}
- Conservation at k gives $x_{jk} = x_{ki}$, and the flows of the two networks correspond, with the same value

Definition 3: residual network

Let $\boldsymbol{x}$ *be a feasible flow. The* ***residual network*** $G(\boldsymbol{x}) = (N, A(\boldsymbol{x}))$ *has, for every arc* $(i,j) \in A$*:*

- *the* ***forward arc*** (i,j)*, with* ***residual capacity*** $r_{ij} = u_{ij} - x_{ij}$*, if* $x_{ij} < u_{ij}$*;*
- *the* ***backward arc*** (j,i)*, with residual capacity* $r_{ji} = x_{ij}$*, if* $x_{ij} > 0$*.*

Remarks on the residual network

- The residual capacity r_{ij} is how much flow can still be sent from i to j: by raising x_{ij} on a forward arc, or by lowering x_{ji} on a backward arc.

- An empty arc gives only its forward arc, a saturated arc only its backward arc, and an arc with $0 < x_{ij} < u_{ij}$ both.
- Hence $G(\boldsymbol{x})$ has at most $2m$ arcs, all with positive residual capacity.

- Without opposite arcs in G, every ordered pair of nodes carries at most one residual arc.

Definition 4: augmenting path

Let $\boldsymbol{x}$ *be a feasible flow. An* ***augmenting path*** *is a directed path* P *from* s *to* t *in* $G(\boldsymbol{x})$*. Its* ***bottleneck*** *is*

$$\delta = \min_{(i,j)\in P} r_{ij} > 0.$$

Augmenting $\boldsymbol{x}$ *along* P *means sending* δ *units along* P*: for every arc of* P*,*

- *if it is a forward arc* (i,j)*, then* $x_{ij} := x_{ij} + \delta$*;*
- *if it is a backward arc* (j,i)*, with* $(i,j) \in A$*, then* $x_{ij} := x_{ij} - \delta$*.*

Lemma 2: augmentation

Augmenting a feasible flow $\boldsymbol{x}$ of value v along an augmenting path P with bottleneck δ gives a feasible flow $\boldsymbol{x}'$ of value $v+\delta$.

- We check the three properties of a feasible flow of value $v + \delta$: the bounds, the conservation, and the value

Proof (1/2)

- The nodes of P are distinct and G has no opposite arcs, so every arc of A changes at most once

1. *Bounds.* On a forward arc,

$$x_{ij} + \delta \leq x_{ij} + r_{ij} = u_{ij};$$

on a backward arc,

$$x_{ij} - \delta \geq x_{ij} - r_{ji} = 0.$$

Proof (2/2)

2. *Conservation.* Let $k \notin \{s, t\}$ be a node of P.
   - The arc of P entering k raises the net inflow of k by δ: a forward arc (h, k) raises x_{hk}, a backward arc (h, k) lowers x_{kh}.
   - In the same way, the arc of P leaving k raises the net outflow of k by δ.
   - The two changes cancel, and the nodes outside P are not touched.

3. *Value.* The path P leaves s once and never enters it again, so the net outflow of s grows by δ.

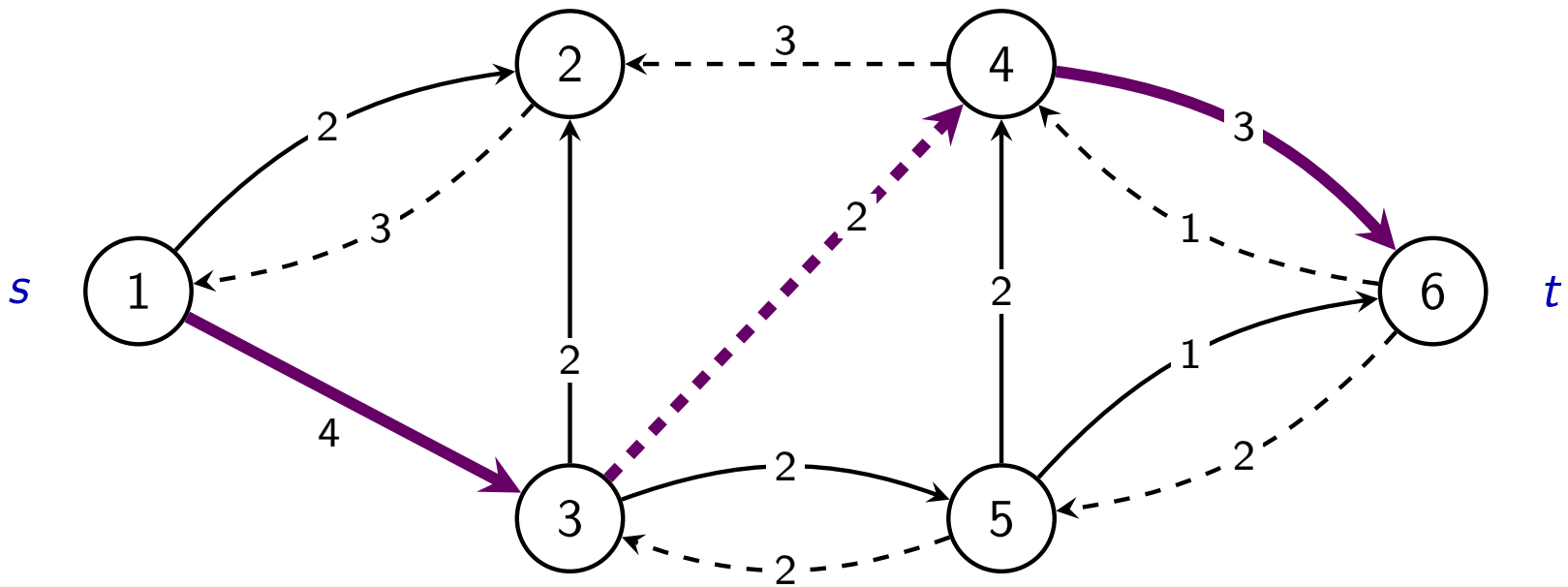
□

Example: the residual network of $\bar{\boldsymbol{x}}$ (1/5)

- For the flow $\bar{\boldsymbol{x}}$ of the reference network, of value 3, the residual network $G(\bar{\boldsymbol{x}})$ has 13 arcs
- The six backward arcs are $(2,1)$, $(3,4)$, $(4,2)$, $(5,3)$, $(6,4)$, $(6,5)$
- The arc $(2,4)$ is saturated, so it gives only the backward arc $(4,2)$, with $r_{42} = \bar{x}_{24} = 3$
- The arc $(1,2)$ has $0 < \bar{x}_{12} = 3 < 5$, so it gives the forward arc $(1,2)$, with $r_{12} = 2$, and the backward arc $(2,1)$, with $r_{21} = 3$

Example: the residual network of $\bar{x}$ (2/5)

The residual network $G(\bar{x})$: each arc carries r_{ij}, the backward arcs are dashed, and the augmenting path found next is violet.



Example: the residual network of $\bar{x}$ (3/5)

- The depth-first search of Chapter 1 from node 1, with the residual arcs of each node by increasing index of the head, labels the nodes $1, 2, 3, 4, 6$:
  - from 2 the only residual arc goes back to 1;
  - from 3 the search takes the backward arc $(3, 4)$ (2 is labelled);
  - from 4 the arc $(4, 6)$.

- The augmenting path is $P = 1, 3, 4, 6$ (violet), with bottleneck

$$\delta = \min\{r_{13},\ r_{34},\ r_{46}\} = \min\{4,\ 2,\ 3\} = 2.$$

Example: the residual network of $\bar{x}$ (4/5)

- Augmenting gives

$$x_{13} = 0 + 2 = 2, \qquad x_{43} = 2 - 2 = 0, \qquad x_{46} = 1 + 2 = 3,$$

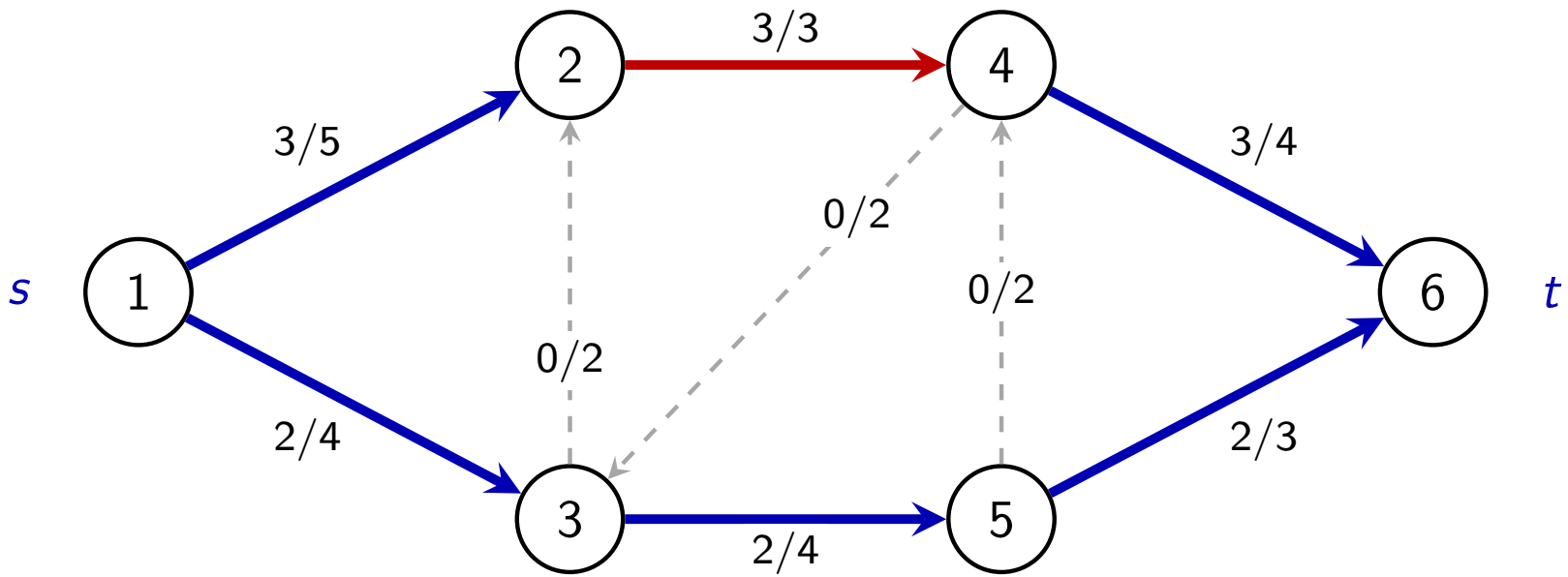
  where x_{43} decreases along the backward arc $(3, 4)$; the value is $3 + 2 = 5$

- The two units that went from 4 to 3 now go from 4 to 6

- Node 3 receives two new units from 1, which continue along $3, 5, 6$

Example: the residual network of $\bar{x}$ (5/5)

The flow after the augmentation, of value 5 (each arc carries x_{ij}/u_{ij}):



Are the backward arcs really needed?

The next example shows what happens if the flows can only grow.

Example: why the backward arcs are needed (1/3)

Suppose that from $\bar{\boldsymbol{x}}$ we use only the forward arcs of the residual network, so that flows can only grow.

- The depth-first search on the forward arcs finds the path $1, 3, 5, 4, 6$, with

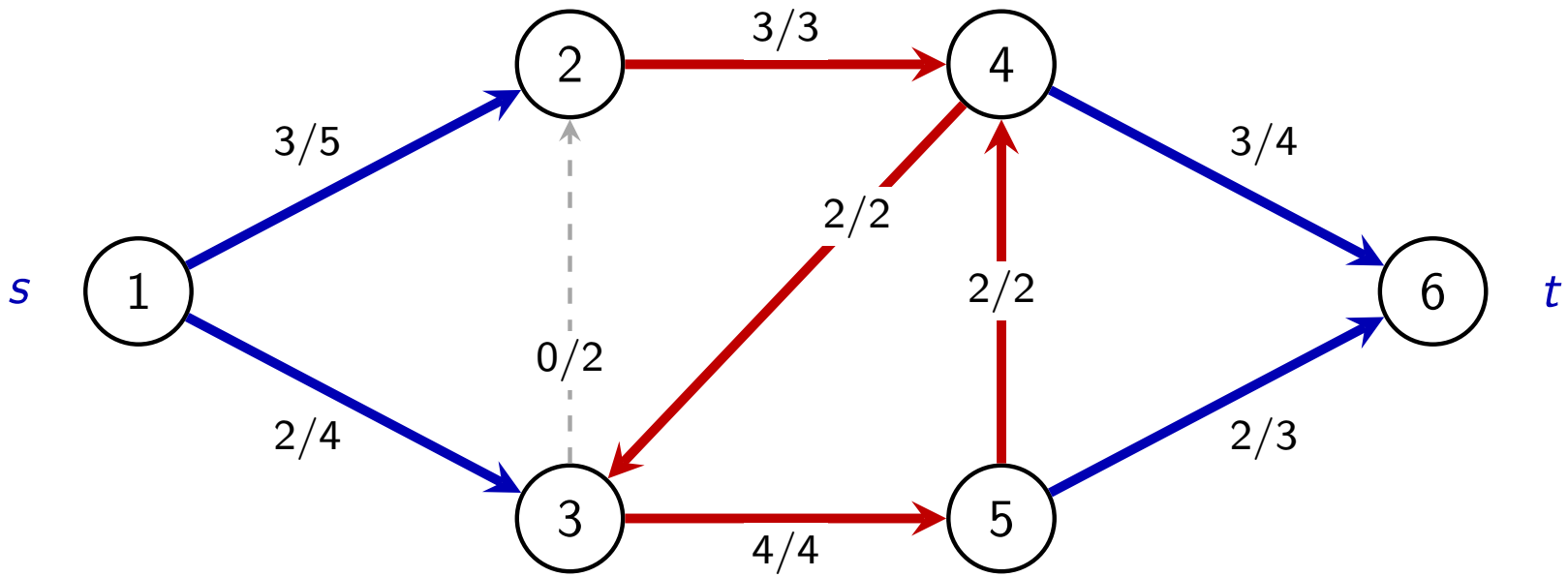
$$\delta = \min\{4, 2, 2, 3\} = 2$$

- The value becomes 5 with the flow

$$\boldsymbol{x} = (3, 2, 3, 0, 4, 2, 3, 2, 2)$$

Example: why the backward arcs are needed (2/3)

The flow $\boldsymbol{x}$ of value 5 (each arc carries x_{ij}/u_{ij}):



- Now the forward arcs $(2, 4)$ and $(3, 5)$ leaving $\{1, 2, 3\}$ are saturated: the search labels only $1, 2, 3$, and the method stops with value 5

Example: why the backward arcs are needed (3/3)

- The value is not maximum: for $S = \{1, 2, 3\}$, the lemma on the flow across a cut gives

$$v = x(\delta^+(S)) - x(\delta^-(S)) = 7 - 2 = 5$$

- The two units on the backward arc $(4, 3)$ of the cut, sent there by the first choices, come back into S

- Only the backward arc $(3, 4)$ of the residual network can cancel them: from this flow, the augmenting paths

$$1, 3, 4, 5, 6 \qquad \text{and} \qquad 1, 3, 4, 6,$$

the first with the backward arcs $(3, 4)$ and $(4, 5)$, with $\delta = 1$ each, reach the value 7

Augmenting along the residual network

- The residual network has a forward arc where the flow can grow and a backward arc where it can shrink
- Sending the bottleneck δ along a path from s to t of the residual network keeps the flow feasible and raises its value by δ
- The backward arcs undo earlier choices

Summary

The residual network $G(\boldsymbol{x})$:

- Forward arc (i, j) with $r_{ij} = u_{ij} - x_{ij}$ if $x_{ij} < u_{ij}$
- Backward arc (j, i) with $r_{ji} = x_{ij}$ if $x_{ij} > 0$

Augmenting paths:

- A path from s to t in $G(\boldsymbol{x})$, with bottleneck $\delta = \min r_{ij}$
- Augmenting: $+\delta$ on forward arcs, $-\delta$ on the arcs of the backward ones; the value grows by δ

Backward arcs: without them the method can stop at value 5 on the reference network

The max-flow min-cut theorem

Introduction: The max-flow min-cut theorem

- In this section we prove the central result of the chapter: a flow is maximum exactly when no augmenting path exists
- The maximum value of a flow equals the minimum capacity of a cut: the bound of weak duality is always attained
- The proof also tells us how to read a minimum cut from a maximum flow

Theorem 1: max-flow min-cut

Let $\boldsymbol{x}$ be a feasible flow of value v. The following statements are equivalent:

(a) *$\boldsymbol{x}$ is a maximum flow;*

(b) *the residual network $G(\boldsymbol{x})$ has no augmenting path;*

(c) *$v = u(S, N \setminus S)$ for some s–t cut $(S, N \setminus S)$.*

Moreover, a maximum flow exists, and the maximum value of a flow equals the minimum capacity of an s–t cut:

$$\tilde{v} = \min \left\{ u(S, N \setminus S) \ : \ S \subseteq N, \ s \in S, \ t \notin S \right\}.$$

How the proof goes

- We prove the chain (a) $\Rightarrow$ (b) $\Rightarrow$ (c) $\Rightarrow$ (a)
- (a) $\Rightarrow$ (b) uses the augmentation lemma, and (c) $\Rightarrow$ (a) the weak duality corollary
- The heart is (b) $\Rightarrow$ (c): the nodes reachable from s in the residual network give the cut
- The existence of a maximum flow comes from Linear Programming

Proof (1/3)

(a) implies (b). An augmenting path would give, by the augmentation lemma, a feasible flow of value $v + \delta > v$.

(b) implies (c). Let S be the set of the nodes reachable from s in $G(\boldsymbol{x})$; then $s \in S$ and, by (b), $t \notin S$.

1. If an arc $(i, j) \in \delta^+(S)$ had $x_{ij} < u_{ij}$, the forward arc (i, j) would be in $G(\boldsymbol{x})$, and j would be reachable: hence $x_{ij} = u_{ij}$.
2. If an arc $(i, j) \in \delta^-(S)$ had $x_{ij} > 0$, the backward arc (j, i), with $j \in S$, would be in $G(\boldsymbol{x})$, and i would be reachable: hence $x_{ij} = 0$.
3. By the weak duality corollary, $v = u(S, N \setminus S)$.

Proof (2/3)

(c) implies (a). By the weak duality corollary.

Existence and equality.

- The feasible region of the MF model is nonempty ($\boldsymbol{x} = 0$, $v = 0$) and bounded:

$$0 \leq x_{ij} \leq u_{ij}, \qquad |v| \leq \sum_{(i,j)\in A} u_{ij}$$

- So the model has an optimal solution (Chapter 15 «The geometry of Linear Programming»)

Proof (3/3)

- For a maximum flow, (a) implies (c): some cut has capacity $\tilde{v}$
- By the weak duality corollary no cut has a smaller capacity

Hence

$$\tilde{v} = \min \left\{ u(S, N \setminus S) \ : \ S \subseteq N, \ s \in S, \ t \notin S \right\}.$$

□

What the theorem gives

- The proof of (b) implies (c) is constructive: at a maximum flow, the nodes reachable from s in the residual network form a minimum cut.
- A flow and a cut with $v = u(S, N \setminus S)$ certify each other: the cut proves that no flow is better, and the flow proves that no cut is smaller.

Example: the maximum flow of the reference network (1/3)

On the reference network the flow

arc	$(1,2)$	$(1,3)$	$(2,4)$	$(3,2)$	$(3,5)$	$(4,3)$	$(4,6)$	$(5,4)$	$(5,6)$
$\tilde{x}_{ij}$	3	4	3	0	4	0	4	1	3
u_{ij}	5	4	3	2	4	2	4	2	3

is feasible, with value

$$\tilde{x}_{12} + \tilde{x}_{13} = 3 + 4 = 7.$$

Example: the maximum flow of the reference network (2/3)

- For $S = \{1, 2\}$ the forward arcs $(1, 3)$, $(2, 4)$ are saturated and the backward arc $(3, 2)$ is empty, so

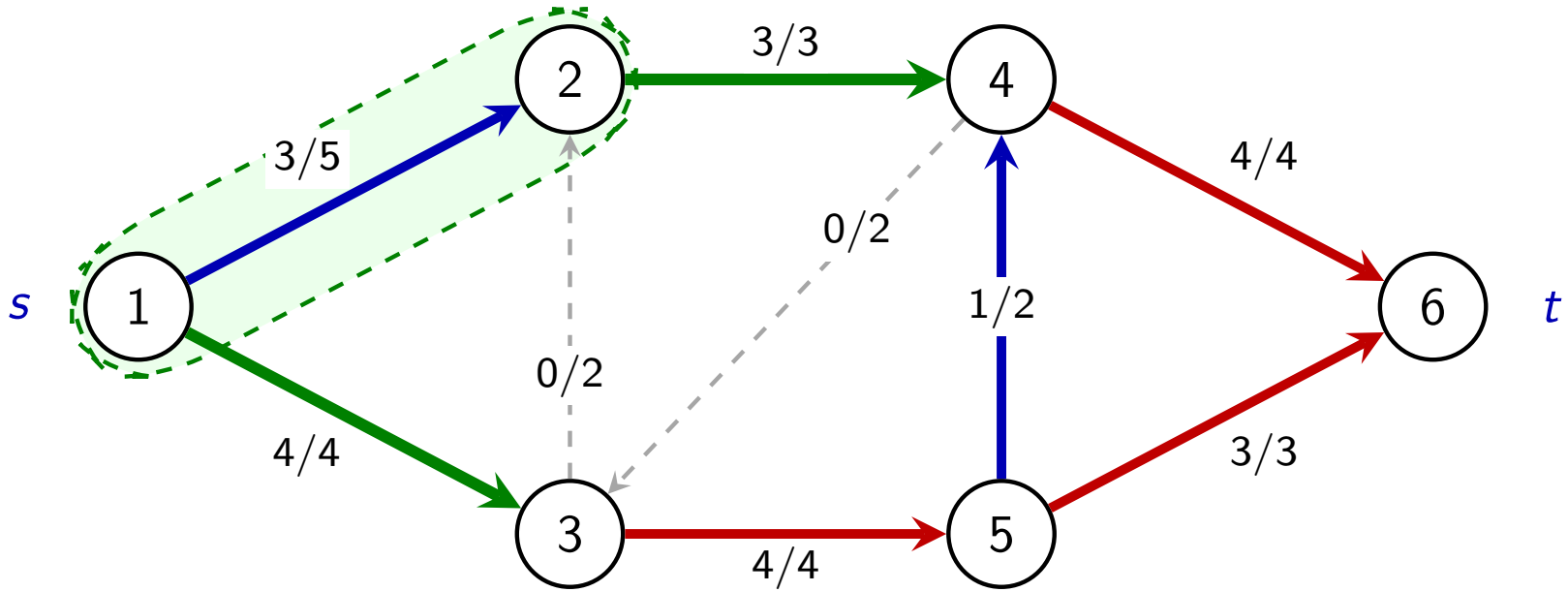
$$u(S, N \setminus S) = u_{13} + u_{24} = 4 + 3 = 7 = v$$

- By the max-flow min-cut theorem, $\tilde{\boldsymbol{x}}$ is a maximum flow, $\tilde{v} = 7$, and $(\{1, 2\}, \{3, 4, 5, 6\})$ is a minimum cut

- In $G(\tilde{\boldsymbol{x}})$ the nodes reachable from 1 are exactly 1 and 2

Example: the maximum flow of the reference network (3/3)

In the figure, S is shaded and the forward arcs of the cut are green.



Uniqueness of the maximum flow (1/2)

- By the weak duality corollary, every maximum flow saturates the forward arcs and empties the backward arcs of each of the three minimum cuts of the reference network:

$$\begin{aligned} S = \{1,2\}: &\quad x_{13} = 4,\ x_{24} = 3,\ x_{32} = 0, \\ S = \{1,2,3\}: &\quad x_{35} = 4,\ x_{43} = 0, \\ S = \{1,2,3,4,5\}: &\quad x_{46} = 4,\ x_{56} = 3. \end{aligned}$$

Uniqueness of the maximum flow (2/2)

- Conservation at nodes 1 and 4 then gives

$$x_{12} = 7 - x_{13} = 3, \qquad x_{54} = x_{43} + x_{46} - x_{24} = 1$$

- Hence $\tilde{\boldsymbol{x}}$ is the unique maximum flow
- while the minimum cut is not unique

Observation 2: saturated arcs and cuts

The saturated arcs of a maximum flow do not form a cut in general.

- *Under* $\tilde{\boldsymbol{x}}$ *the saturated arcs are* $(1,3)$, $(2,4)$, $(3,5)$, $(4,6)$, $(5,6)$, *with total capacity* 18.
- *A minimum cut is read from the nodes reachable from* s *in the residual network.*

Maximum flows and minimum cuts

- A flow is maximum if and only if its residual network has no augmenting path, if and only if its value equals the capacity of a cut
- The maximum value equals the minimum cut capacity
- The nodes reachable from s in the residual network of a maximum flow form a minimum cut

Summary

The theorem:

- Maximum flow $\Leftrightarrow$ no augmenting path $\Leftrightarrow$ $v = u(S, N \setminus S)$ for some cut
- $\tilde{v}$ equals the minimum capacity of an s–t cut

Certificates:

- A flow and a cut with the same value certify each other
- The minimum cut is read from the nodes reachable from s, not from the saturated arcs

Reference network: $\tilde{v} = 7$, unique maximum flow $\tilde{\boldsymbol{x}}$, minimum cut $S = \{1, 2\}$

The Ford–Fulkerson algorithm

Introduction: The Ford–Fulkerson algorithm

- In this section we turn the max-flow min-cut theorem into an algorithm
- We prove that it is correct with integer capacities, that it gives an integer maximum flow, and we bound its running time
- We run it on the reference network, iteration by iteration, and we see that its number of augmentations can grow with the capacities

The idea

- By the max-flow min-cut theorem, a flow can be improved as long as its residual network has an augmenting path
- The **Ford–Fulkerson algorithm** starts from the zero flow and augments along such paths until none is left

The Ford–Fulkerson algorithm

Algorithm: Ford–Fulkerson algorithm

Input : a directed graph $G = (N, A)$ with integer capacities $u_{ij} \geq 0$, a source s and a sink t

Output : a maximum flow $\boldsymbol{x}$ and a minimum cut $(S, N \setminus S)$

1 **Step 0 (initialization).** Set $x_{ij} := 0$ for every $(i, j) \in A$, and $v := 0$.

2 **Step 1 (search).** Build the residual network $G(\boldsymbol{x})$ and search it from s.

3 If t is not labelled, `STOP`: $\boldsymbol{x}$ is a maximum flow, and S is the set of the labelled nodes.

4 **Step 2 (augmentation).** Let P be the path of the predecessors from s to t, and $\delta := \min_{(i,j) \in P} r_{ij}$.

5 Augment $\boldsymbol{x}$ along P by δ, set $v := v + \delta$, and go to Step 1.

The search of Step 1

- Step 1 can use any search of Chapter 1, Section «Graph search»
- In this module the Ford–Fulkerson algorithm uses the **depth-first search**:
  - *LIST* is a stack;
  - the residual arcs leaving each node are scanned by increasing index of the head;
  - the search stops as soon as t is labelled.
- When the search does not label t, it has labelled all the nodes reachable from s (Chapter 1, Theorem «correctness of the search»): they form the set S of Step 1

Theorem 2: correctness of Ford–Fulkerson

Let the capacities be integer. Then every flow computed by the Ford–Fulkerson algorithm is integer, the algorithm stops after at most $\tilde{v}$ augmentations, and at the stop $\boldsymbol{x}$ is a maximum flow and $(S, N \setminus S)$ is a minimum cut.

- The proof uses the augmentation lemma and the max-flow min-cut theorem

Proof

1. *Integrality.* The zero flow is integer. If $\boldsymbol{x}$ is integer, all residual capacities are integer, so δ is a positive integer and the augmented flow is integer.

2. *Termination.* Every flow is feasible, and every augmentation raises the value by $\delta \geq 1$. No value exceeds $\tilde{v}$, so there are at most $\tilde{v}$ augmentations.

3. *Optimality.* At the stop t is not labelled, so $G(\boldsymbol{x})$ has no augmenting path and S is the set of the nodes reachable from s. The proof of (b) implies (c) gives $v = u(S, N \setminus S)$: the flow is maximum and the cut is minimum.

□

Corollary 2: integrality

If all the capacities are integer, there is a maximum flow with integer entries.

Proof

- The flow returned by the Ford–Fulkerson algorithm is integer and maximum, by the correctness theorem of Ford–Fulkerson.

□

Two remarks on integrality

- The same conclusion follows from Chapter 4, Theorem «integer vertices»: with the arc (t, s), the MF model is a minimum cost flow model with integer data, and its optimum is attained at an integer vertex.

- Rational capacities become integer after multiplication by a common denominator, which multiplies every flow and every cut by the same factor: the algorithm stops also with rational capacities.

Theorem 3: running time of Ford–Fulkerson

With integer capacities, the Ford–Fulkerson algorithm runs in $O((n+m)\,\tilde{v})$ *time, and*

$$\tilde{v} \leq (n-1)\,U,$$

where U *is the largest capacity.*

Proof

- Every iteration
  - builds $G(\boldsymbol{x})$ in $O(m)$ time,
  - searches it in $O(n+m)$ time, since $G(\boldsymbol{x})$ has at most $2m$ arcs (Chapter 1, Theorem «running time of the search»),
  - and changes at most $n-1$ flows along P.
- There are at most $\tilde{v}$ iterations with an augmentation, plus the last one.

- By the weak duality corollary with $S = \{s\}$, $\tilde{v} \leq u(\{s\}, N \setminus \{s\})$, and s has at most $n-1$ leaving arcs, each with capacity at most U.

□

A pseudopolynomial algorithm

- The bound grows with the value U of the capacities, and not only with the number of their digits, about $\log_2 U$
- The running time is **pseudopolynomial**
- The example on the number of augmentations, later in this section, shows that the number of augmentations can really grow with the capacities

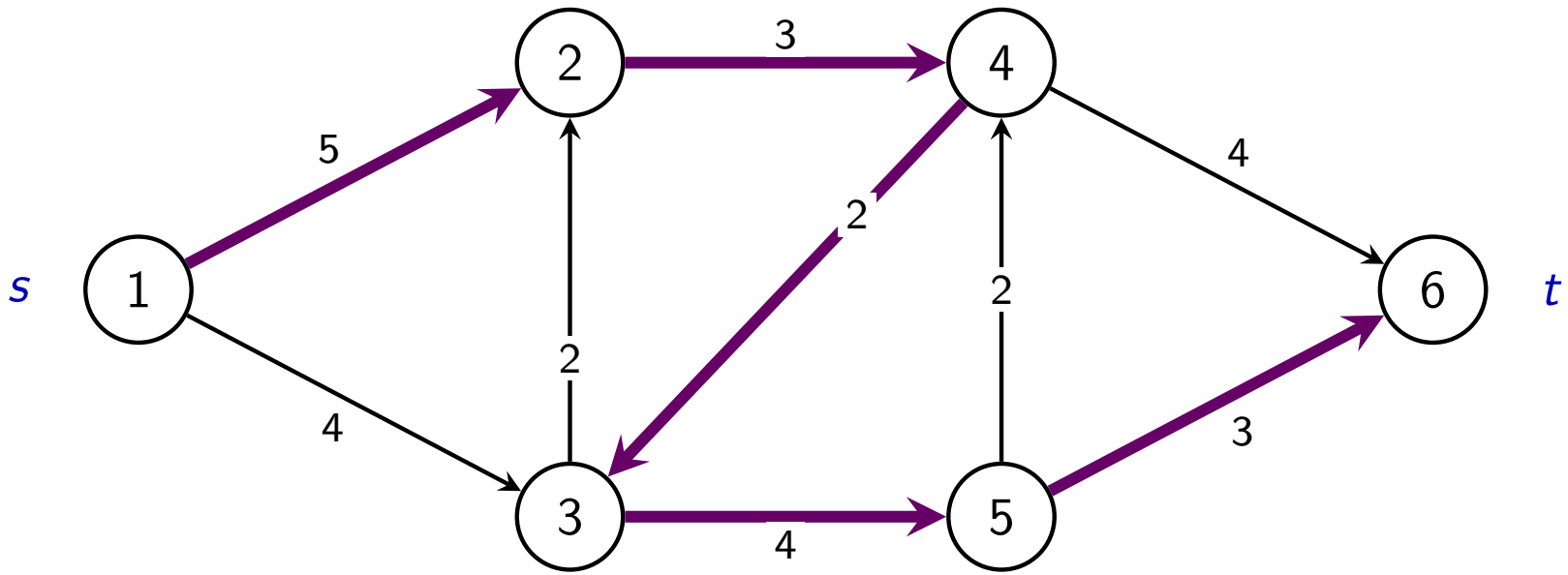
Example: Ford–Fulkerson on the reference network (1/9)

We run the algorithm on the reference network. Each row is an iteration: the augmenting path, the residual capacities of its arcs, the backward arcs it uses, the bottleneck and the new value.

iteration	augmenting path	r_{ij} on P	backward arcs	δ	v
1	$1, 2, 4, 3, 5, 6$	$5, 3, 2, 4, 3$	–	2	2
2	$1, 2, 4, 6$	$3, 1, 4$	–	1	3
3	$1, 3, 4, 6$	$4, 2, 3$	$(3, 4)$	2	5
4	$1, 3, 5, 4, 6$	$2, 2, 2, 1$	–	1	6
5	$1, 3, 5, 6$	$1, 1, 1$	–	1	7

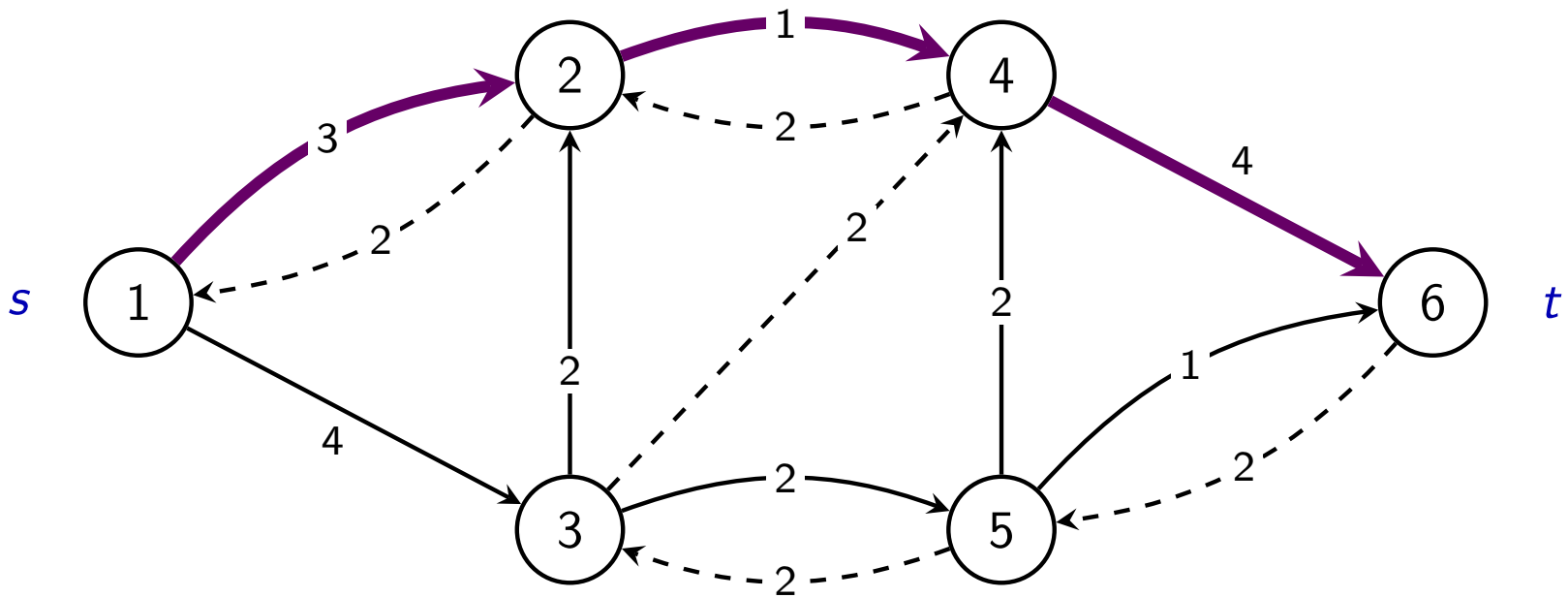
Example: Ford–Fulkerson on the reference network (2/9)

Iteration 1. All the arcs are empty, $G(0) = G$; the path $1, 2, 4, 3, 5, 6$ (violet) has $\delta = 2$.



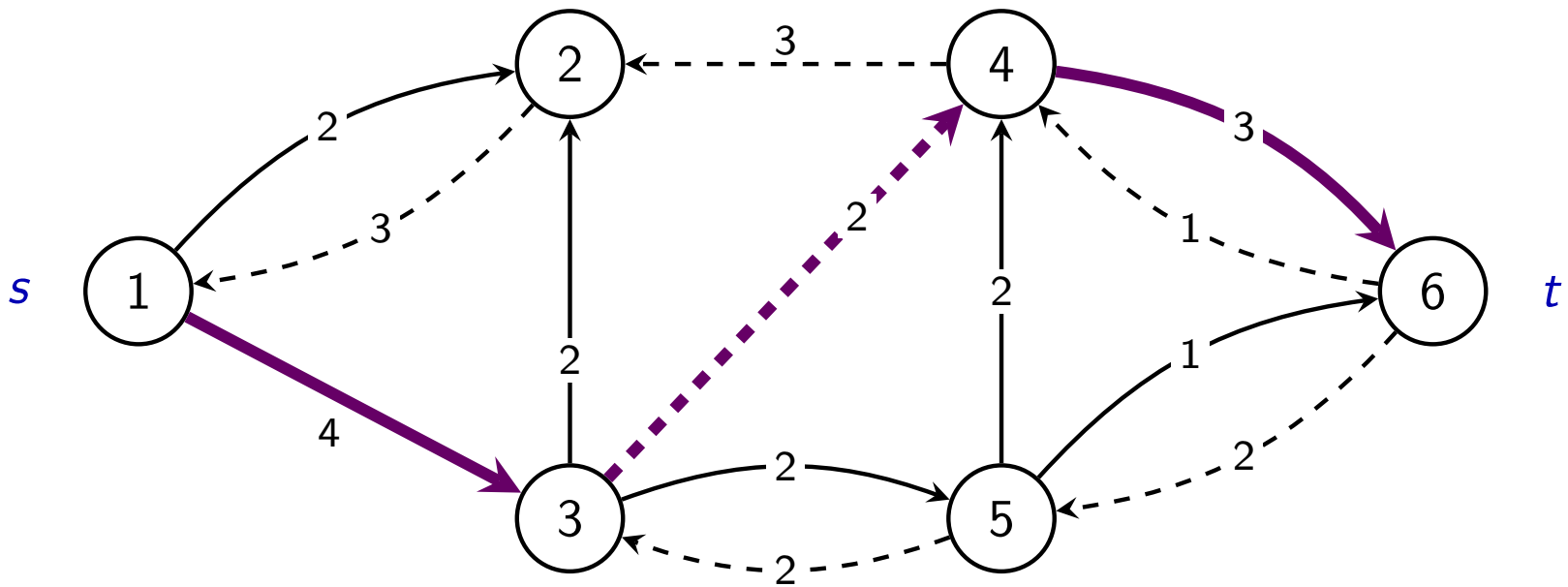
Example: Ford–Fulkerson on the reference network (3/9)

Iteration 2. The residual network of the flow of value 2; the path $1, 2, 4, 6$ has $\delta = 1$.



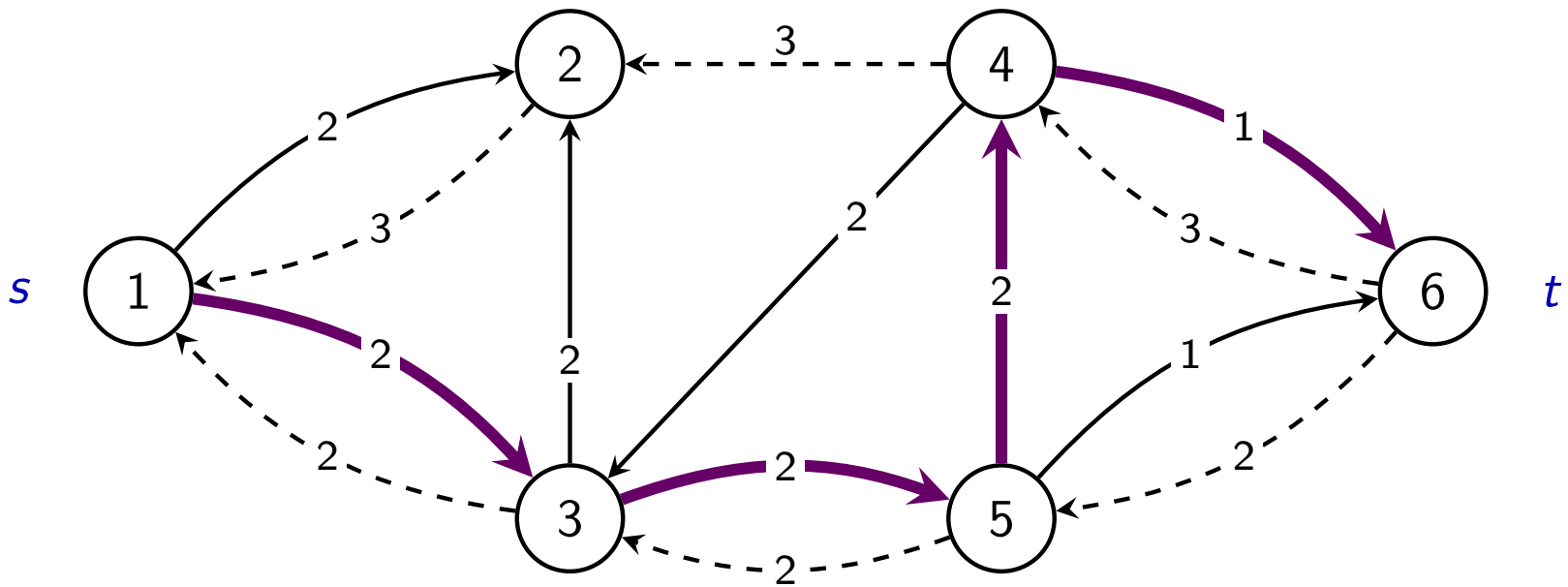
Example: Ford–Fulkerson on the reference network (4/9)

Iteration 3. The flow is $\bar{x}$; the path $1, 3, 4, 6$ uses the backward arc $(3, 4)$, $\delta = 2$.



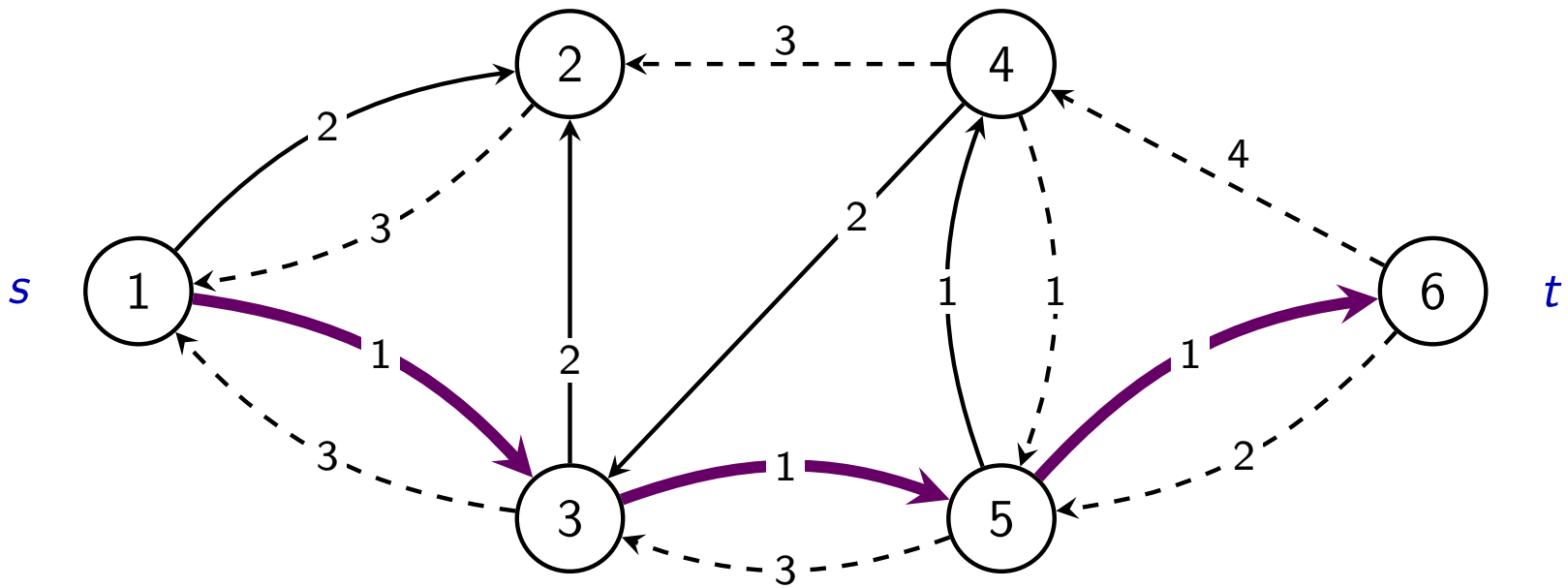
Example: Ford–Fulkerson on the reference network (5/9)

Iteration 4. The residual network of the flow of value 5; the path $1, 3, 5, 4, 6$ has $\delta = 1$.



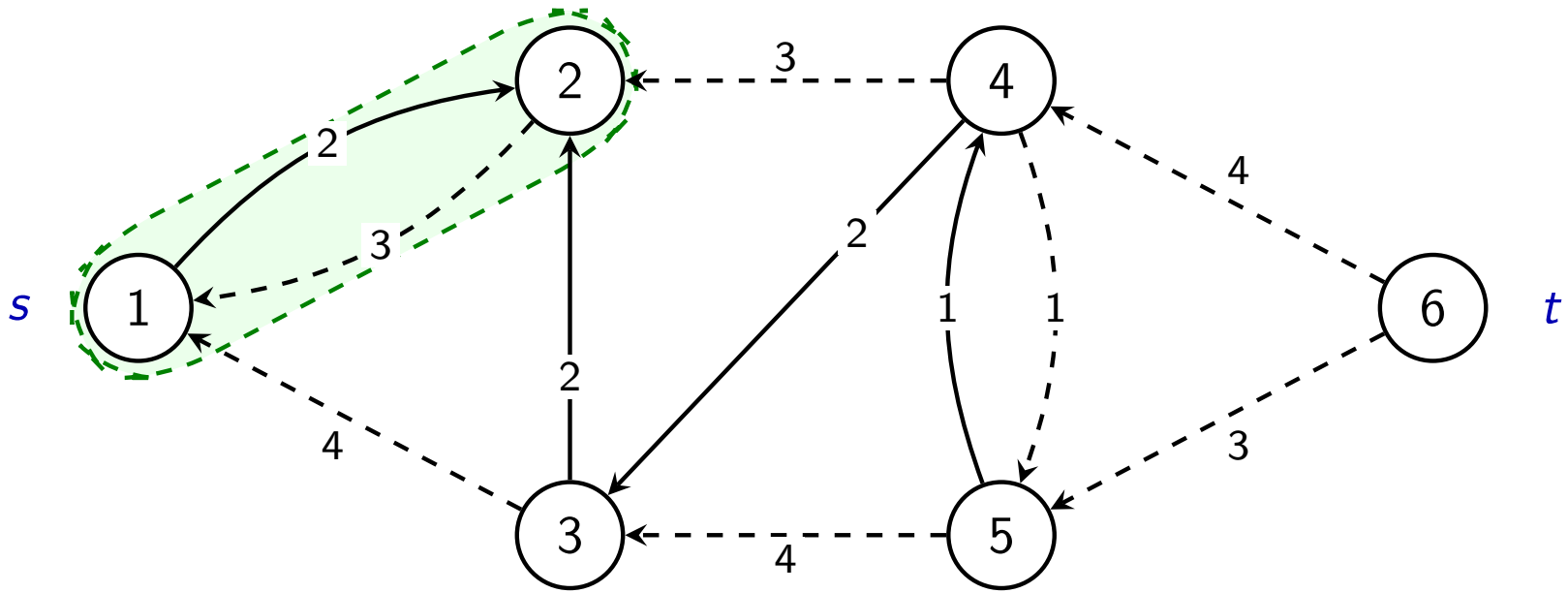
Example: Ford–Fulkerson on the reference network (6/9)

Iteration 5. The residual network of the flow of value 6; the path $1, 3, 5, 6$ has $\delta = 1$.



Example: Ford–Fulkerson on the reference network (7/9)

Sixth search. The residual network of the flow of value 7: the search labels only 1 and 2 (shaded), and the algorithm stops.



Example: Ford–Fulkerson on the reference network (8/9)

The flows after each iteration ($k = 0$ is the zero flow) are:

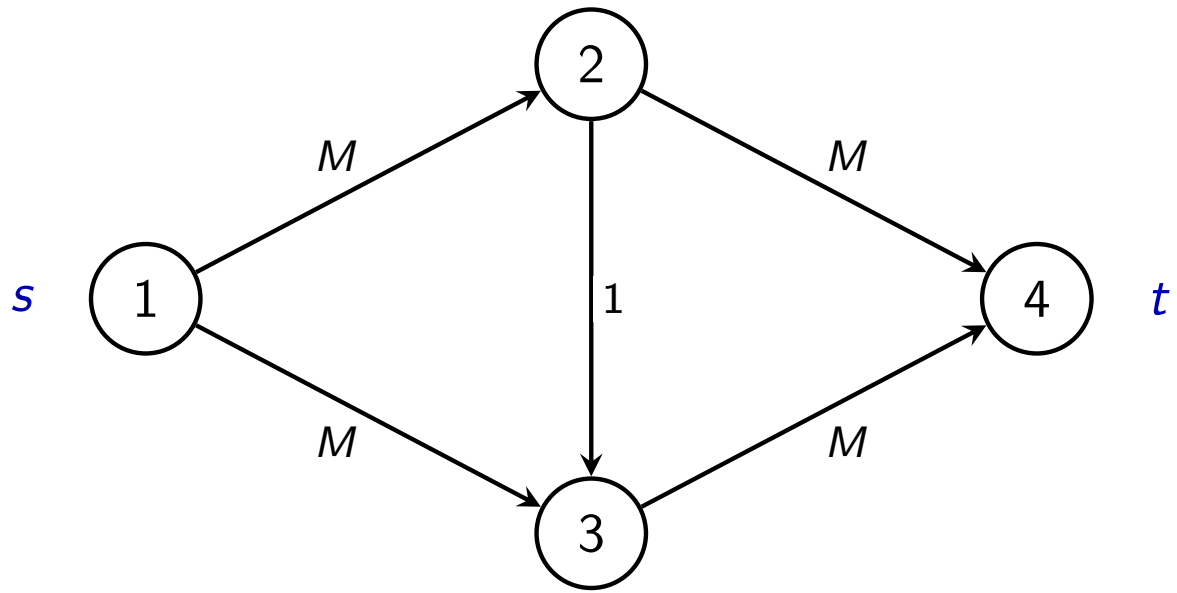
k	x_{12}	x_{13}	x_{24}	x_{32}	x_{35}	x_{43}	x_{46}	x_{54}	x_{56}
0	0	0	0	0	0	0	0	0	0
1	2	0	2	0	2	2	0	0	2
2	3	0	3	0	2	2	1	0	2
3	3	2	3	0	2	0	3	0	2
4	3	3	3	0	3	0	4	1	2
5	3	4	3	0	4	0	4	1	3

Example: Ford–Fulkerson on the reference network (9/9)

- Iteration 1 is the depth-first search of Chapter 1, Example «DFS from node 1»: all the arcs are empty, $G(0) = G$, and the path goes through the arc $(4, 3)$.
- After iteration 2 the flow is $\bar{\boldsymbol{x}}$ of the reference network, and iteration 3 is the augmentation of the example on the residual network of $\bar{\boldsymbol{x}}$: the backward arc $(3, 4)$ cancels the two units sent on $(4, 3)$ in iteration 1.
- The sixth search labels only the nodes 1 and 2, and the algorithm stops: the flow is the maximum flow $\tilde{\boldsymbol{x}}$, and $S = \{1, 2\}$ gives the minimum cut of capacity $u_{13} + u_{24} = 7$.
- The algorithm makes 5 augmentations, within the bound $\tilde{v} = 7$ of the correctness theorem.

Example: many augmentations (1/3)

- In the network below, M is a positive integer
- The paths $1, 2, 4$ and $1, 3, 4$ carry M units each, and the cut $S = \{1\}$ has capacity $2M$, so $\tilde{v} = 2M$



Example: many augmentations (2/3)

- The Ford–Fulkerson algorithm allows any augmenting path
- If the paths $1, 2, 3, 4$ and $1, 3, 2, 4$ alternate, the second one through the backward arc $(3, 2)$, every bottleneck is $\delta = 1$, given by the arc $(2, 3)$ or by its backward arc
- Then $2M$ augmentations are needed: two million for $M = 10^6$

Example: many augmentations (3/3)

- With the depth-first search of this module, for $M \geq 2$, the paths are

$$1, 2, 3, 4, \qquad 1, 2, 4, \qquad 1, 3, 2, 4, \qquad 1, 3, 4,$$

with

$$\delta = 1, \qquad M - 1, \qquad 1, \qquad M - 1$$

- Four augmentations

The Ford–Fulkerson algorithm in a nutshell

- The Ford–Fulkerson algorithm augments along paths of the residual network until none is left
- With integer capacities it stops after at most $\tilde{v}$ augmentations, with an integer maximum flow and a minimum cut
- Its running time $O((n+m)\,\tilde{v})$ is pseudopolynomial

Summary

The algorithm:

- Start from $\boldsymbol{x} = 0$; search $G(\boldsymbol{x})$ from s (depth-first search); augment by δ; repeat
- At the stop, the labelled nodes S give a minimum cut

Its properties:

- Integer capacities: integer flows, at most $\tilde{v}$ augmentations, an integer maximum flow exists
- Running time $O((n+m)\,\tilde{v})$ with $\tilde{v} \leq (n-1)\,U$: pseudopolynomial

Reference network: 5 augmentations, $\delta = 2, 1, 2, 1, 1$, $\tilde{v} = 7$

The Edmonds–Karp algorithm

Introduction: The Edmonds–Karp algorithm

- In this section we change only the search of Step 1: the breadth-first search instead of the depth-first search
- The augmenting paths then have the fewest arcs, and the number of augmentations no longer depends on the capacities
- We prove that the algorithm makes at most $n\,m$ augmentations: it is strongly polynomial

Definition 5: Edmonds–Karp algorithm

*The **Edmonds–Karp algorithm** is the Ford–Fulkerson algorithm in which Step 1 uses the **breadth-first search**:*

- *LIST is a queue;*
- *the residual arcs leaving each node are scanned by increasing index of the head;*
- *the search stops as soon as t is labelled.*

The capacities are any non-negative real numbers.

Shortest augmenting paths and distances

- By Chapter 1, Proposition «BFS finds shortest paths in number of arcs», the augmenting path has the minimum number of arcs among the paths from s to t in $G(\boldsymbol{x})$.
- For a feasible flow $\boldsymbol{x}$ and a node i, $d_{\boldsymbol{x}}(i)$ is the minimum number of arcs of a path from s to i in $G(\boldsymbol{x})$, and $d_{\boldsymbol{x}}(i) = +\infty$ if i is not reachable.

Critical arcs

- An arc of the augmenting path is **critical** if its residual capacity equals δ: after the augmentation its residual capacity is zero, and it leaves the residual network.
- An augmentation along P lowers r_{ij} by δ and raises r_{ji} by δ on every arc (i, j) of P, and changes no other residual capacity.
- Hence the arcs that enter the residual network are reverses of arcs of P.

Lemma 3: the distances do not decrease

Let $\boldsymbol{x}'$ be obtained from $\boldsymbol{x}$ by an augmentation along a path P from s to t of $G(\boldsymbol{x})$ with the minimum number of arcs. Then $d_{\boldsymbol{x}'}(i) \geq d_{\boldsymbol{x}}(i)$ for every $i \in N$.

- In the proof we first show that $d_{\boldsymbol{x}}$ grows by at most 1 along every arc of $G(\boldsymbol{x}')$, and then we follow a shortest path of $G(\boldsymbol{x}')$

Proof (1/3)

We show that every arc (a, b) of $G(\boldsymbol{x}')$ satisfies

$$d_{\boldsymbol{x}}(b) \leq d_{\boldsymbol{x}}(a) + 1.$$

1. If (a, b) is an arc of $G(\boldsymbol{x})$, a shortest path from s to a followed by (a, b) reaches b with $d_{\boldsymbol{x}}(a) + 1$ arcs.

Proof (2/3)

2. Otherwise (a, b) is the reverse of an arc (b, a) of P.
   - The part of P from s to each of its nodes has the minimum number of arcs: a path to that node with fewer arcs, followed by the rest of P, would give a walk from s to t with fewer arcs than P, and a path inside it (Chapter 1, Lemma «a walk contains a path»).
   - Hence

$$d_{\boldsymbol{x}}(a) = d_{\boldsymbol{x}}(b) + 1, \quad \text{and} \quad d_{\boldsymbol{x}}(b) \leq d_{\boldsymbol{x}}(a) + 1.$$

Proof (3/3)

- Now let $s = i_0, i_1, \ldots, i_q = i$ be a path of $G(\boldsymbol{x}')$ with $q = d_{\boldsymbol{x}'}(i)$ arcs
- Along its arcs $d_{\boldsymbol{x}}$ grows by at most 1 at a time, so

$$d_{\boldsymbol{x}}(i) \leq d_{\boldsymbol{x}}(s) + q = d_{\boldsymbol{x}'}(i)$$

- If i is not reachable in $G(\boldsymbol{x}')$, then $d_{\boldsymbol{x}'}(i) = +\infty$

□

Lemma 4: critical arcs

In the Edmonds–Karp algorithm, let the residual arc (i, j) be critical in the augmentation at the flow $\boldsymbol{x}$, and critical again in a later augmentation at the flow $\boldsymbol{y}$. Then $d_{\boldsymbol{y}}(i) \geq d_{\boldsymbol{x}}(i) + 2$.

- In the proof we apply the lemma «the distances do not decrease» at every augmentation between $\boldsymbol{x}$ and $\boldsymbol{y}$

Proof (1/2)

1. At $\boldsymbol{x}$, the arc (i, j) lies on a path with the minimum number of arcs, so
$$d_{\boldsymbol{x}}(j) = d_{\boldsymbol{x}}(i) + 1;$$
after the augmentation $r_{ij} = 0$.

2. At $\boldsymbol{y}$ the arc (i, j) is back in the residual network, so some augmentation in between, at a flow $\boldsymbol{z}$, raised r_{ij}: its path used the reverse arc (j, i), and
$$d_{\boldsymbol{z}}(i) = d_{\boldsymbol{z}}(j) + 1.$$

Proof (2/2)

3. The distances never decrease, so

$$d_{\boldsymbol{z}}(j) \geq d_{\boldsymbol{x}}(j) \qquad \text{and} \qquad d_{\boldsymbol{y}}(i) \geq d_{\boldsymbol{z}}(i).$$

Hence

$$d_{\boldsymbol{y}}(i) \geq d_{\boldsymbol{z}}(i) = d_{\boldsymbol{z}}(j) + 1 \geq d_{\boldsymbol{x}}(j) + 1 = d_{\boldsymbol{x}}(i) + 2.$$

□

Theorem 4: running time of Edmonds–Karp

For any non-negative capacities, the Edmonds–Karp algorithm stops after at most $n\,m$ augmentations with a maximum flow and a minimum cut, and it runs in $O(n\,m\,(n+m))$ time, that is $O(n\,m^2)$ when $m \geq n-1$.

Proof (1/2)

1. Every augmentation has at least one critical arc: an arc of P whose residual capacity equals δ.

2. When (i, j) is critical, i lies before t on a path from s to t with at most $n - 1$ arcs, so

$$0 \leq d(i) \leq n - 2.$$

   By the lemma on critical arcs this distance grows by at least 2 between two critical times, so (i, j) is critical at most

$$\left\lfloor \frac{n-2}{2} \right\rfloor + 1 \leq \frac{n}{2} \text{ times.}$$

Proof (2/2)

3. Every residual arc is (i, j) or (j, i) for an arc $(i, j) \in A$: there are at most $2m$ of them, and at most

$$2m \cdot \frac{n}{2} = n\,m$$

augmentations.

4. Every iteration takes $O(n + m)$ time, as in the running time theorem of Ford–Fulkerson. At the stop $G(\boldsymbol{x})$ has no augmenting path, and the max-flow min-cut theorem applies as in the proof of the correctness of Ford–Fulkerson.

□

A strongly polynomial algorithm

- The bound depends only on n and m, not on the capacities
- The Edmonds–Karp algorithm is **strongly polynomial**
- It stops also with irrational capacities

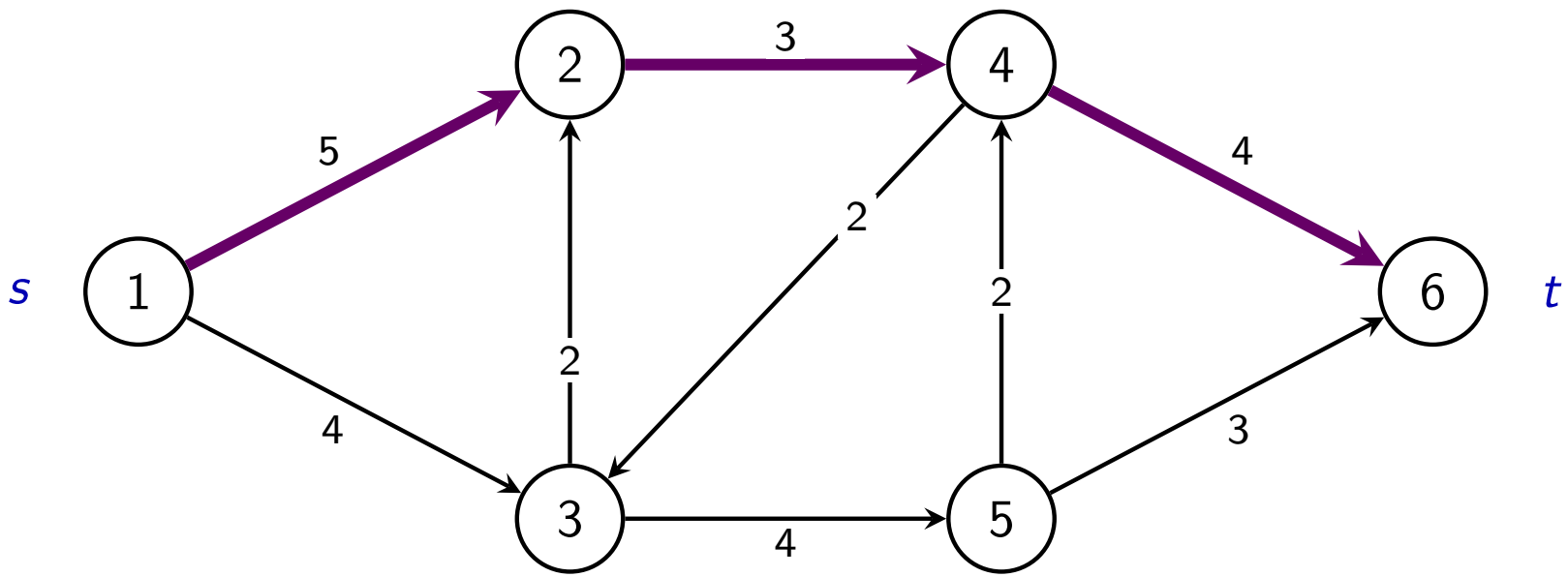
Example: Edmonds–Karp on the reference network (1/7)

We run the algorithm on the reference network; the column «arcs» is the number of arcs of the augmenting path.

iteration	augmenting path	arcs	r_{ij} on P	backward arcs	δ	v
1	$1, 2, 4, 6$	3	$5, 3, 4$	–	3	3
2	$1, 3, 5, 6$	3	$4, 4, 3$	–	3	6
3	$1, 3, 5, 4, 6$	4	$1, 1, 2, 1$	–	1	7

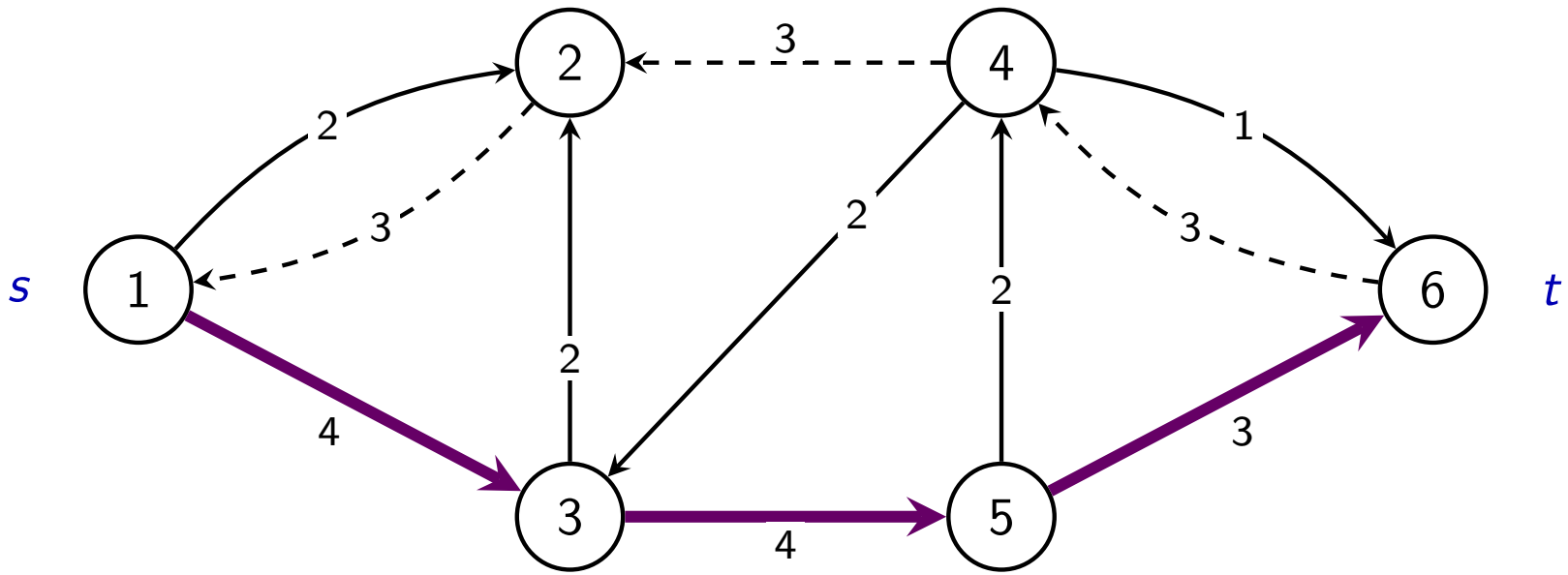
Example: Edmonds–Karp on the reference network (2/7)

Iteration 1. $G(0) = G$; the breadth-first search finds $1, 2, 4, 6$, with $\delta = 3$.



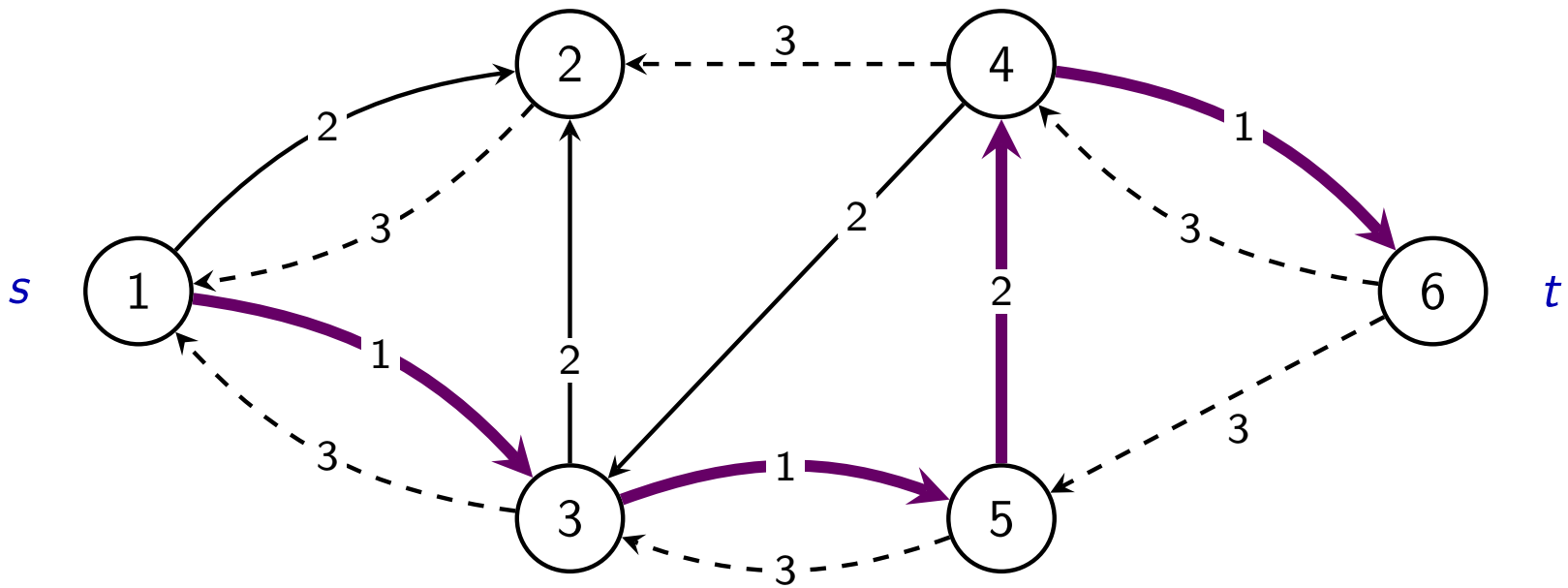
Example: Edmonds–Karp on the reference network (3/7)

Iteration 2. The residual network of the flow of value 3; the path $1, 3, 5, 6$ has $\delta = 3$.



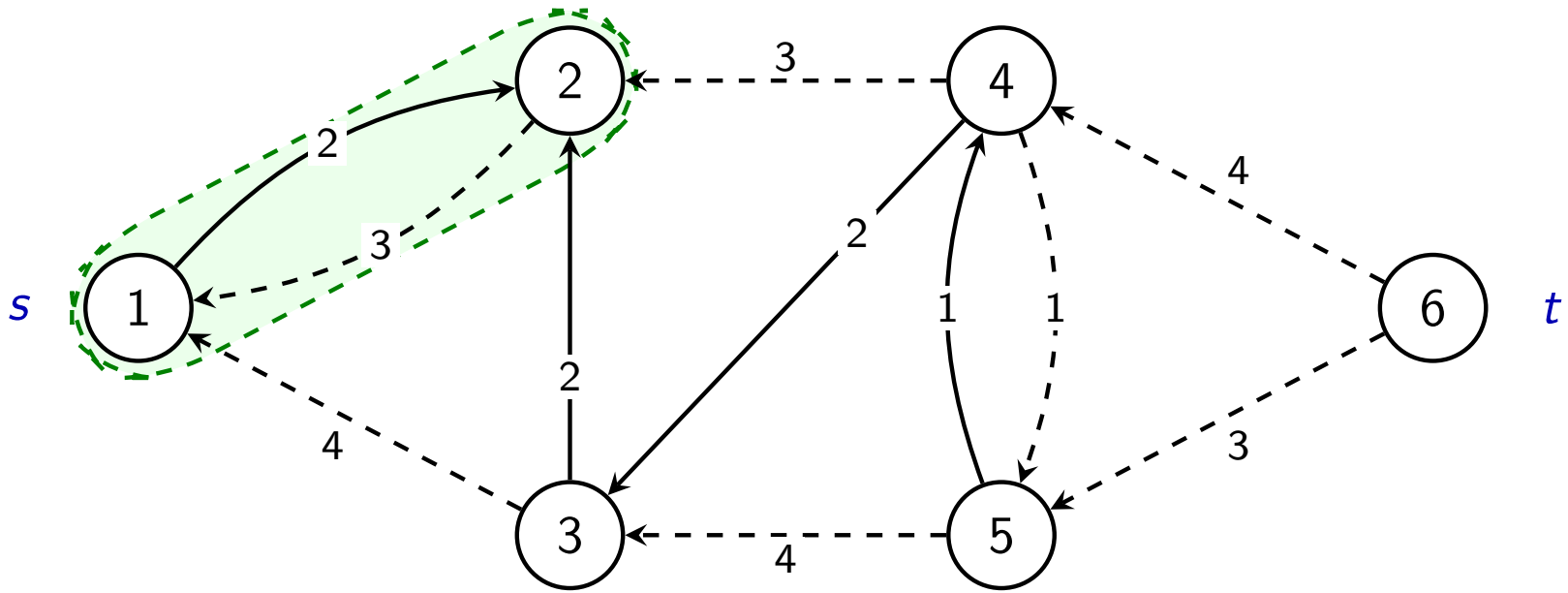
Example: Edmonds–Karp on the reference network (4/7)

Iteration 3. The residual network of the flow of value 6; the path $1, 3, 5, 4, 6$ has $\delta = 1$.



Example: Edmonds–Karp on the reference network (5/7)

Fourth search. The flow is $\tilde{x}$: the search labels only 1 and 2 (shaded), as in Ford–Fulkerson.



Example: Edmonds–Karp on the reference network (6/7)

The distances $d_x(i)$ in the residual network before each search are:

iteration	$d(1)$	$d(2)$	$d(3)$	$d(4)$	$d(5)$	$d(6)$
1	0	1	1	2	2	3
2	0	1	1	3	2	3
3	0	1	1	3	2	4
stop	0	1	∞	∞	∞	∞

Iteration 1 is the breadth-first search of Chapter 1, Example «BFS from node 1», whose tree path to node 6 is $1, 2, 4, 6$.

Example: Edmonds–Karp on the reference network (7/7)

- The critical arcs are $(2, 4)$ in iteration 1, $(5, 6)$ in iteration 2, and $(1, 3)$, $(3, 5)$, $(4, 6)$ in iteration 3.
- When $(2, 4)$ leaves the residual network, $d(4)$ grows from 2 to 3; no distance ever decreases.
- The algorithm needs 3 augmentations, against the 5 of Ford–Fulkerson, far below the bound $n\,m = 54$.
- It ends with the same maximum flow $\tilde{\boldsymbol{x}}$, the unique one, and the same cut $S = \{1, 2\}$.
- On the network of the example with many augmentations the breadth-first search finds $1, 2, 4$ and then $1, 3, 4$, each with $\delta = M$: two augmentations for every M.

The Edmonds–Karp algorithm in a nutshell

- The Edmonds–Karp algorithm augments along paths with the fewest arcs, found by the breadth-first search
- The distances from s in the residual network never decrease, and every arc is critical at most $\frac{n}{2}$ times
- The algorithm makes at most $n\,m$ augmentations: $O(n\,m^2)$ time, for any capacities

Summary

The algorithm:

- Ford–Fulkerson with the breadth-first search: augmenting paths with the fewest arcs

The analysis:

- The distances $d_{\boldsymbol{x}}(i)$ never decrease
- Between two critical times of (i, j), $d(i)$ grows by at least 2
- At most $n\,m$ augmentations, $O(n\,m^2)$ time: strongly polynomial

Reference network: 3 augmentations, $\delta = 3, 3, 1$, the same $\tilde{\boldsymbol{x}}$ and cut $S = \{1, 2\}$

Applications

Introduction: Applications

- In this section we solve two combinatorial problems as maximum flow problems with unit capacities
- The maximum matching in a bipartite graph: assign jobs to workers
- The maximum number of arc-disjoint paths from s to t (Menger's theorem)
- The integrality corollary is the key: it gives 0/1 flows, which are matchings and paths

Definition 6: bipartite graph and matching (1/2)

*An undirected graph $H = (V_1 \cup V_2, E)$ is **bipartite** if V_1 and V_2 are disjoint and every edge has one endpoint in V_1 and one in V_2; we write the edges as $\{i, j\}$ with $i \in V_1$ and $j \in V_2$.*

- *A **matching** is a set $M \subseteq E$ of edges, no two of which have a common endpoint.*
- *The **maximum matching problem** asks for a matching with the largest number of edges.*

Definition: bipartite graph and matching (2/2)

*The **network of** H has*

- *the nodes* $\{s\} \cup V_1 \cup V_2 \cup \{t\}$,
- *the arcs* (s, i) *for* $i \in V_1$, (i, j) *for every edge* $\{i, j\} \in E$, (j, t) *for* $j \in V_2$,
- *and all capacities equal to* 1.

- With V_1 a set of workers, V_2 a set of jobs, and an edge when a worker can do a job, a matching assigns jobs to workers, at most one job per worker and one worker per job.
- In the order of the nodes, s comes first and t last; the other nodes keep their indices.

Proposition 1: matchings and flows

The maximum number of edges of a matching of H equals the maximum value $\tilde{v}$ of a flow in the network of H. If $\tilde{\boldsymbol{x}}$ is an integer maximum flow, then $M = \{\{i,j\} \in E : \tilde{x}_{ij} = 1\}$ is a maximum matching.

- In the proof we go from matchings to flows, from integer flows to matchings, and we close with the integrality corollary

Proof (1/3)

1. *From matchings to flows.*
   - Given a matching M, set $x = 1$ on the arcs (s, i), (i, j), (j, t) of every edge $\{i, j\} \in M$, and $x = 0$ elsewhere.
   - Every node of $V_1 \cup V_2$ is an endpoint of at most one edge of M, so the flow is feasible, and its value is $|M|$.

Proof (2/3)

2. *From integer flows to matchings.* Let $\boldsymbol{x}$ be an integer feasible flow, so $x_{ij} \in \{0, 1\}$.
   - A node $i \in V_1$ has the single entering arc (s, i), so at most one arc (i, j) carries flow.
   - A node $j \in V_2$ has the single leaving arc (j, t), so at most one arc (i, j) carries flow.
   - Hence $M = \{\{i, j\} : x_{ij} = 1\}$ is a matching.
   - For $S = \{s\} \cup V_1$, the forward arcs are the arcs (i, j) and there are no backward arcs: by the lemma on the flow across a cut, $v = |M|$.

Proof (3/3)

3. By the integrality corollary there is an integer maximum flow $\tilde{\boldsymbol{x}}$.
   - By 1, no matching has more than $\tilde{v}$ edges,
   - and by 2 the matching of $\tilde{\boldsymbol{x}}$ has $\tilde{v}$ edges.

□

Example: workers and jobs (1/3)

Four workers $V_1 = \{1, 2, 3, 4\}$ and three jobs $V_2 = \{5, 6, 7\}$ have the edges

$$E = \big\{\{1,5\}, \{1,6\}, \{2,5\}, \{3,6\}, \{3,7\}, \{4,6\}\big\}.$$

The Edmonds–Karp algorithm on the network of H, with the nodes in the order $s, 1, 2, \ldots, 7, t$, gives:

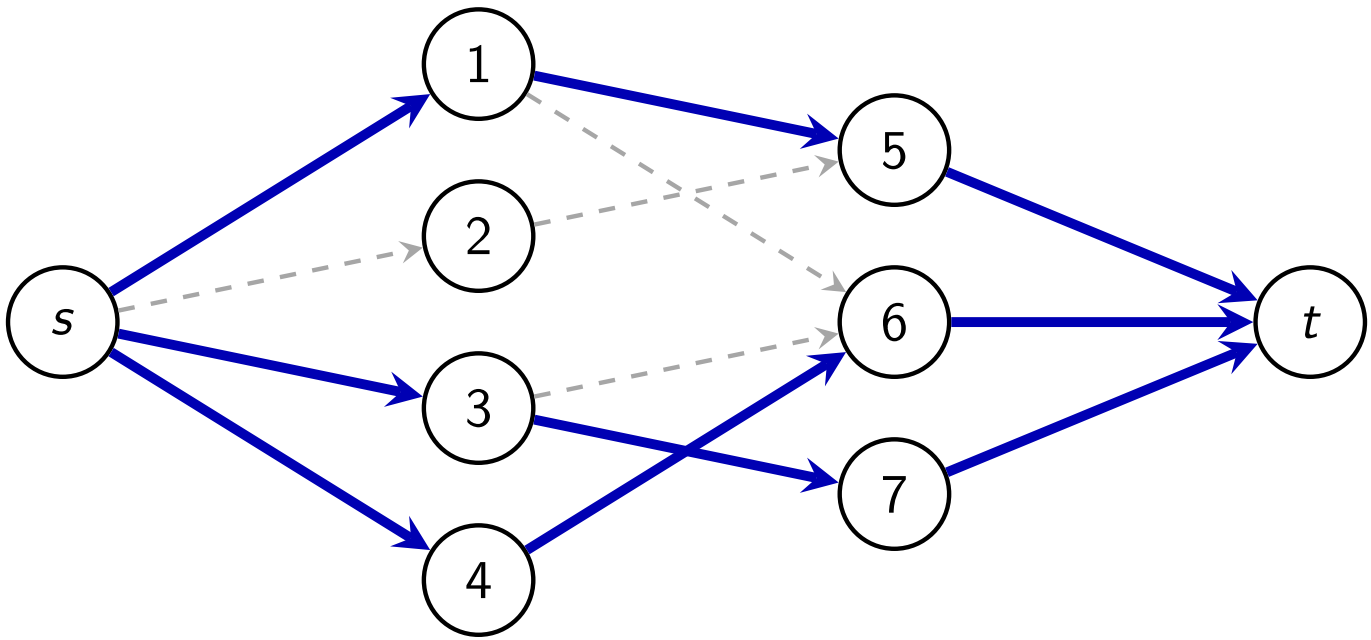
iteration	augmenting path	r_{ij} on P	backward arcs	δ	v
1	$s, 1, 5, t$	$1, 1, 1$	–	1	1
2	$s, 3, 6, t$	$1, 1, 1$	–	1	2
3	$s, 4, 6, 3, 7, t$	$1, 1, 1, 1, 1$	$(6, 3)$	1	3

Example: workers and jobs (2/3)

- Iteration 3 uses the backward arc $(6, 3)$: worker 3 leaves job 6 for job 7, and worker 4 takes job 6
- The maximum matching is $M = \{\{1, 5\}, \{3, 7\}, \{4, 6\}\}$, with 3 edges; worker 2 stays without a job
- At the stop the search labels $S = \{s, 1, 2, 4, 5, 6\}$; the forward arcs of the cut are $(s, 3)$, $(5, t)$, $(6, t)$, with capacity $3 = |M|$

Example: workers and jobs (3/3)

In the figure the arcs with flow 1 are blue.



Arc-disjoint paths

- Two directed paths are **arc-disjoint** if they have no common arc; they may share nodes
- How many arc-disjoint paths from s to t can we find? And how many arcs must be removed to disconnect t from s?
- The next theorem says that the two numbers are equal, and the proof is a maximum flow argument

Theorem 5: Menger, arc version

Let $G = (N, A)$ be a directed graph and $s \neq t$ two nodes. The maximum number of pairwise arc-disjoint directed paths from s to t equals the minimum number of arcs whose removal leaves no directed path from s to t.

- In the proof we give every arc the capacity 1, and we let $\tilde{v}$ be the maximum value of a flow

Proof (1/4)

Give every arc the capacity 1, and let $\tilde{v}$ be the maximum value of a flow.

1. *Paths give flows.*
   - The sum of the incidence vectors of k arc-disjoint paths from s to t is a feasible flow of value k: every arc carries at most 1, and every path enters and leaves its inner nodes.
   - Hence $k \leq \tilde{v}$.

Proof (2/4)

2. *Flows give paths.* By the integrality corollary there is a flow $\boldsymbol{x}$ with $x_{ij} \in \{0, 1\}$ and value $\tilde{v}$. If $\tilde{v} \geq 1$, walk from s along arcs with $x_{ij} = 1$, each used at most once, until t is reached:
   - at a node $i \notin \{s, t\}$ the walk has used one entering arc more than leaving arcs, and conservation gives an unused leaving arc with flow;
   - at s the walk has used as many leaving arcs as entering ones, while the arcs with flow leaving s are $\tilde{v} \geq 1$ more than those entering: an unused one exists.

Proof (3/4)

- The walk ends, at t, and it contains a path P from s to t (Chapter 1, Lemma «a walk contains a path»)
- Setting $x_{ij} = 0$ on P leaves a 0/1 flow of value $\tilde{v} - 1$
- Repeating gives $\tilde{v}$ arc-disjoint paths

Proof (4/4)

3. *Removing arcs.*
   - If removing a set F of arcs leaves no path from s to t, the set S of the nodes still reachable from s has $t \notin S$ and $\delta^+(S) \subseteq F$, so
   $$|F| \geq u(S, N \setminus S) \geq \tilde{v}.$$
   - Conversely, every path from s to t leaves S through an arc of $\delta^+(S)$: removing the $\tilde{v}$ forward arcs of a minimum cut (max-flow min-cut theorem) leaves no path.

□

Example: arc-disjoint paths in the reference graph (1/2)

- On the reference graph with all capacities 1, $s = 1$ and $t = 6$, the Edmonds–Karp algorithm finds the paths

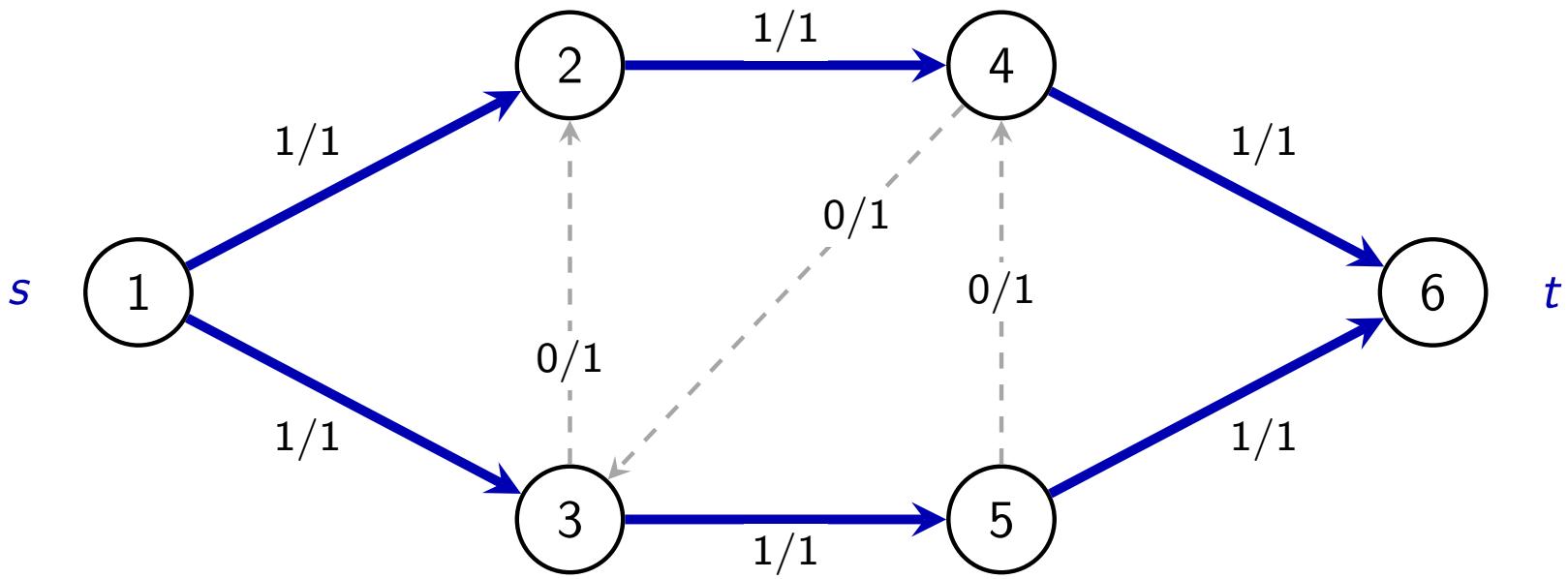
$$1, 2, 4, 6 \qquad \text{and} \qquad 1, 3, 5, 6,$$

  and $\tilde{v} = 2$

- These two paths are arc-disjoint
- Removing the two arcs $(1, 2)$, $(1, 3)$ leaving node 1 leaves no path from 1 to 6: both numbers of Menger's theorem are 2

Example: arc-disjoint paths in the reference graph (2/2)

The two arc-disjoint paths $1, 2, 4, 6$ and $1, 3, 5, 6$ are blue; the other arcs are empty.



Matchings and paths as flows

- A maximum matching of a bipartite graph is an integer maximum flow in a network with unit capacities
- So are the arc-disjoint paths from s to t
- The minimum cut certifies both

Summary

Bipartite matching:

- Network of H: $s \to V_1 \to V_2 \to t$, all capacities 1
- Maximum matching = integer maximum flow; in the example $|M| = 3$

Arc-disjoint paths (Menger):

- Maximum number of arc-disjoint paths = minimum number of arcs to remove = $\tilde{v}$ with unit capacities
- In the reference graph: the paths $1, 2, 4, 6$ and $1, 3, 5, 6$

Typical mistakes

Introduction: Typical mistakes

- In this section we list the most common mistakes on cuts, residual networks and augmenting paths
- They concern the capacity of a cut, the backward arcs, the bottleneck, the reading of the minimum cut and the running times
- Recognizing them helps to run the algorithms correctly by hand

Observation 3: typical mistakes (1/2)

- ***Counting the backward arcs of a cut.*** *The capacity $u(S, N \setminus S)$ adds up only the arcs from S to $N \setminus S$.*
- ***Forgetting the backward arcs of the residual network.*** *With forward arcs only, the method can stop at a flow that is not maximum, as at value 5 on the reference network.*
- ***Giving a backward arc the capacity u_{ij}.*** *The backward arc (j, i) has $r_{ji} = x_{ij}$: only the flow already sent can be sent back.*

Observation: typical mistakes (2/2)

- ***Taking the bottleneck from the capacities.*** *The bottleneck is the smallest residual capacity r_{ij} on the path, not the smallest u_{ij}.*
- ***Adding δ on a backward arc.*** *Along a backward arc (j, i) the flow x_{ij} decreases by δ.*
- ***Reading the cut from the saturated arcs.*** *The minimum cut is given by the nodes reachable from s in the residual network at the stop; the saturated arcs need not form a cut.*
- ***Calling Ford–Fulkerson polynomial.*** *Its number of augmentations can grow with the capacities, as $2M$ in the example with many augmentations; Edmonds–Karp makes at most $n\,m$.*

Summary

Cuts and residual networks:

- Only the forward arcs count in the capacity of a cut
- The backward arcs of the residual network are needed, with $r_{ji} = x_{ij}$

Augmenting:

- The bottleneck is the smallest r_{ij} on the path; on a backward arc the flow decreases by δ

At the stop:

- The minimum cut comes from the reachable nodes; Ford–Fulkerson is only pseudopolynomial

In summary

Introduction: In summary

- We collect the key ideas of the chapter in a few points
- We close with the reference network at a glance: the numbers computed in this chapter

Summary (1/3)

- A feasible flow respects the capacities and is conserved at the nodes other than s and t; its value is the net outflow of s, and it equals the net flow across every s–t cut.
- Weak duality: $v \leq u(S, N \setminus S)$ for every flow and every cut.
- The residual network has forward arcs with $r_{ij} = u_{ij} - x_{ij}$ and backward arcs with $r_{ji} = x_{ij}$; an augmenting path raises the value by its bottleneck δ.

Summary (2/3)

- Max-flow min-cut: a flow is maximum if and only if no augmenting path exists, and $\tilde{v}$ equals the minimum cut capacity; with integer capacities there is an integer maximum flow.
- Ford–Fulkerson (depth-first search): with integer capacities at most $\tilde{v}$ augmentations, pseudopolynomial. Edmonds–Karp (breadth-first search): at most $n\,m$ augmentations, $O(n\,m^2)$ time.
- Maximum matchings in bipartite graphs and arc-disjoint paths are integer maximum flows.

Summary (3/3)

The reference network at a glance.

capacities, $s = 1$, $t = 6$	$\boldsymbol{u} = (5, 4, 3, 2, 4, 2, 4, 2, 3)$
maximum flow (unique)	$\tilde{\boldsymbol{x}} = (3, 4, 3, 0, 4, 0, 4, 1, 3)$, $\tilde{v} = 7$
minimum cuts, capacity 7	$S = \{1, 2\}$, $\{1, 2, 3\}$, $\{1, 2, 3, 4, 5\}$
Ford–Fulkerson (DFS)	5 augmentations, $\delta = 2, 1, 2, 1, 1$
Edmonds–Karp (BFS)	3 augmentations, $\delta = 3, 3, 1$
all capacities 1	arc-disjoint paths $1, 2, 4, 6$ and $1, 3, 5, 6$